



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**

Membre de
HONORIS UNITED UNIVERSITIES



Ecole Marocaine des Sciences de l'Ingénieur

Honoris United Universities — EMSI Rabat

Filière : Intelligence Artificielle & Science des Données

4^e Année — 4IASD

Rapport de Stage

▷ **Projet *AutoFlow* v1.1 — prototype supervise**

AutoFlow : Système d'Automatisation Intelligent des Workflows d'Entreprise

Cas d'application : agence de location de voitures — Rabat

Intitulé du stage : *AI Engineer — automatisation des workflows métier*

Encadrant académique : *Nom de l'encadrant à compléter*

Tuteur entreprise : *Nom du tuteur à compléter*

Entreprise d'accueil : *Structure d'accueil à compléter*

Réalisé par :

Nom et prénom à compléter

Soutenue publiquement le : _____

Remerciements

Je tiens en premier lieu à exprimer ma profonde gratitude à l'ensemble du corps enseignant de l'**Ecole Marocaine des Sciences de l'Ingenieur (EMSI)**, dont la rigueur pédagogique et l'exigence intellectuelle ont forgé les compétences mobilisées tout au long de ce projet. Le parcours en Intelligence Artificielle et Science des Données m'a offert un socle solide, tant sur les fondamentaux mathématiques que sur les technologies émergentes de l'écosystème IA.

J'adresse mes remerciements les plus sincères à mon **encadrant académique**, _____, pour la qualité de son accompagnement, sa disponibilité et la pertinence de ses orientations méthodologiques. Ses relectures critiques et ses questions exigeantes ont considérablement structuré la démarche d'ingénierie de ce travail.

Ma reconnaissance va également à mon **tuteur entreprise**, _____, ainsi qu'à l'ensemble de l'équipe de l'agence d'accueil, qui a accepté d'ouvrir ses opérations quotidiennes à l'observation. Leur confiance et la transparence de leurs échanges ont rendu possible la reconstruction fidèle du contexte métier et la validation successive des hypothèses d'audit.

Je remercie enfin mes camarades de promotion IASD pour les échanges techniques, les sessions de revue de code et l'atmosphère collaborative qui ont enrichi ce travail, ainsi que ma famille pour son soutien indéfectible tout au long de ce cursus.

L'auteur

Resume

Les petites et moyennes entreprises de services au Maroc reçoivent leurs demandes clients par des canaux dispersés — WhatsApp, réseaux sociaux, téléphone, formulaire, comptoir — et les traitent manuellement. Il en résulte des re-saisies, des vérifications lentes, des relances oubliées et une faible visibilité pour le dirigeant. Ce rapport de stage étudie ce problème dans le cas d'une **agence de location de voitures à Rabat** et propose **AutoFlow**, un système d'automatisation *supervise* qui structure chaque demande, vérifie la disponibilité selon des règles déterministes, prépare réponses et relances, trace chaque décision, et transmet à une personne tout cas ambigu ou sensible.

L'architecture est volontairement compacte : un **orchestrateur LangGraph** coordonne **trois agents spécialisés** — *comprendre, vérifier, préparer* — séparés par un **point de contrôle humain** obligatoire pour toute décision commerciale. Elle privilégie la fiabilité à l'autonomie : aucune disponibilité n'est inventée (vérification déterministe), aucun message n'est envoyé sans clic humain (`interrupt()` + `Command(resume)`), tout indicateur est calculé depuis un journal d'événements auditable. L'extraction d'information est *gardée par la preuve* (*evidence gate*) : un champ produit par un LLM est rejeté s'il ne cite pas un passage exact du message source.

Le prototype livre comporte un **espace d'administration** en dix écrans, un **jeu de données fictives contrôlées** de V1 (24 véhicules, 80 réservations et 45 messages sur six semaines) rejoue à travers le vrai graphe, et **sept simulations métier** automatisées. Sur 45 messages étiquetés, l'agent Intake atteint **82–100 % d'exactitude selon le champ**. Dans cet échantillon, les **8/8 extractions comportant une erreur sont routées vers une revue humaine**. Ces chiffres de validation ne sont pas des gains métier ; les indicateurs d'impact sont encadrés pour une mesure avant/après en pilote.

Mots-cles : systèmes multi-agents, LangGraph, human-in-the-loop, orchestration de workflows, extraction d'information, evidence-gated LLM, PME, automatisation supervisée.

Abstract

Moroccan service SMEs receive customer requests through scattered channels — WhatsApp, social networks, phone, form, front desk — and process them by hand. This internal friction produces re-typing, slow checks, forgotten follow-ups and poor managerial visibility. This report studies the problem for a **car-rental agency in Rabat** and proposes **AutoFlow**, a *supervised* workflow-automation system that structures every request, checks availability deterministically, drafts replies and reminders, traces every decision and hands any ambiguous or sensitive case to a human.

The architecture is deliberately compact : one **LangGraph orchestrator** coordinates **three narrow agents** — *understand, verify, prepare* — separated by a mandatory **human-in-the-loop gate**. It favours reliability over autonomy : no availability is fabricated (deterministic verification), nothing is sent without a human click (**interrupt()** + **Command(resume)**), every KPI is derived from an auditable event log. Information extraction is *evidence-gated* : a field produced by an LLM is rejected if it does not cite an exact substring of the source message.

The delivered prototype includes a ten-screen **admin dashboard**, a controlled **fictional V1 dataset** (24 vehicles, 80 bookings and 45 messages over six weeks) replayed through the actual graph, and **seven automated business simulations**. On 45 labelled messages, the Intake agent reaches **82–100 % accuracy depending on the field** ; all **8/8 erroneous extractions in this sample are routed to human review**. These validation results are not business outcomes. Business KPIs are framed for before/after measurement during a pilot, with no figure claimed in advance.

Keywords : multi-agent systems, LangGraph, human-in-the-loop, workflow orchestration, information extraction, evidence-gated LLM, SME, supervised automation.

Note de methodologie et de reproductibilite

Ce manuscrit distingue trois niveaux de preuve. Les **faits implementes** sont traces dans le depot (code, schemas, tests et journal d'evenements). Les **resultats de validation** proviennent d'un jeu de donnees fictives etiquetees et de simulations end-to-end. Les **effets metier** (temps de reponse, conversion, charge de l'equipe) ne sont pas encore mesures : ils constituent l'objet d'un pilote aupres d'une agence.

Le module d'import Excel, ajoute a la V1, permet de charger un fichier d'agence ; ses donnees ne sont pas utilisees pour etablir les metriques du chapitre 7. Cette separation evite de confondre donnees d'interface, donnees de demonstration et donnees d'evaluation. Les commandes de reproduction et la provenance de chaque figure sont documentees en annexe et dans `rapport_assets/README.md`.

Table des matières

Remerciements	1
Resume	2
Abstract	3
Note de methodologie et de reproductibilite	4
Introduction generale	11
1 Contexte General et Problematique	13
1.1 Introduction	13
1.2 L'entreprise d'accueil et son marche	13
1.3 Observation d'une journee-type	14
1.4 Les six frictions operationnelles	15
1.5 Problematique	15
1.6 Objectifs du stage	16
1.6.1 Objectif pedagogique	16
1.6.2 Objectif technique	16
1.6.3 Objectif fonctionnel	16
1.7 Organisation du rapport	17
1.8 Conclusion	17
2 Etat de l'Art et Choix Techniques	18
2.1 Introduction	18
2.2 Automatisation classique : RPA et plateformes low-code	18
2.3 Agents LLM et cadres d'orchestration	19
2.3.1 L'emergence des cadres orientes graphes d'etats	19
2.3.2 LangGraph (LangChain)	19
2.3.3 Microsoft Agent Framework	19
2.3.4 CrewAI et AutoGen	19
2.3.5 Standards emergents : MCP et A2A	20
2.4 Extraction d'information structuree et confiance	20
2.4.1 Le probleme de l'hallucination	20
2.4.2 L'ancrage par la preuve (<i>evidence gate</i>)	20
2.4.3 La calibration de la confiance	20
2.4.4 Inspiration : Jev AI — « probabilites, pas prose »	21
2.5 Human-in-the-loop : de la contrainte a la feature	21

2.6	Synthese : le stack technique retenu	22
2.7	Modele economique pour l'agence	23
2.8	Conclusion	23
3	Analyse des Besoins et Cahier des Charges	24
3.1	Introduction	24
3.2	Identification des acteurs	24
3.3	Exigences fonctionnelles (RF1–RF10)	25
3.4	Exigences non fonctionnelles	26
3.5	Perimetre inclus et exclu	26
3.5.1	Fonctionnalites incluses (V1 demonstrable)	26
3.5.2	Fonctionnalites exclues (<i>audit-gated</i>)	26
3.6	Contraintes et hypotheses	27
3.7	Alignement avec les objectifs pedagogiques	27
3.8	Conclusion	28
4	Conception Architecturale	29
4.1	Introduction	29
4.2	Vue contexte — qui parle au systeme ?	29
4.3	Vue conteneurs — de quoi est fait le systeme ?	30
4.4	Vue composants — le graphe LangGraph	31
4.5	Machine a dix etats et transitions legales	32
4.6	Matrice d'escalade	33
4.7	Vue flux de donnees	34
4.8	Vue sequence — scenario d'ambiguite avec remise	34
4.9	Vue deploiement	35
4.10	Vue defaillances	36
4.11	Conclusion	36
5	Realisation : Agents et Orchestrateur	38
5.1	Introduction	38
5.2	Organisation du code	38
5.3	Agent Intake — comprendre le message	39
5.3.1	Le moteur de regles (toujours execute)	39
5.3.2	L'evidence gate	39
5.3.3	Les signaux typés	39
5.4	Agent Disponibilite — verifier sans LLM	40
5.5	Agent Suivi — preparer sans envoyer	40
5.6	Orchestrateur LangGraph — coordonner et suspendre	40
5.6.1	Structure du graphe	41
5.6.2	Le point d'interruption humain	41
5.6.3	Le checkpointer SQLite	41
5.7	Difficultes rencontrees et resolutions	42

5.8	Conclusion	42
6	Realisation : API, Espace Admin, Integrations	43
6.1	Introduction	43
6.2	API REST FastAPI	43
6.3	Espace d'administration React	44
6.3.1	Vue d'ensemble	44
6.3.2	La file a valider	44
6.3.3	Le suivi en direct et le raisonnement	45
6.3.4	Les simulations et le mode presentation	46
6.4	Donnees realistes et rejeu d'historique	46
6.5	Integrations n8n	47
6.6	Deploiement	47
6.7	Conclusion	47
7	Evaluation et Validation	48
7.1	Introduction	48
7.2	Niveau 1 — Tests d'integration (16 tests verts)	49
7.3	Niveau 2 — Exactitude de l'agent Intake	49
7.3.1	Protocole d'evaluation	49
7.3.2	Le filet de securite	50
7.3.3	La calibration de la confiance	50
7.4	Niveau 2bis — Robustesse	51
7.5	Niveau 3 — Cadre de mesure des indicateurs metier	52
7.6	Limites declarees de l'evaluation	53
7.7	Grille d'evaluation pour le jury	53
7.8	Conclusion	53
8	Discussion, Bilan et Perspectives	54
8.1	Introduction	54
8.2	Ce que le systeme fait bien — et ce qu'il ne fait pas	54
8.2.1	Les forces	54
8.2.2	Les limites assumees	54
8.3	Ou est l'IA dans AutoFlow ?	55
8.4	Conformite reglementaire et ethique	55
8.4.1	Loi 09-08 (protection des donnees personnelles au Maroc)	55
8.4.2	Ethique de l'automatisation	55
8.5	Generalisation — pourquoi le nom AutoFlow	55
8.6	Bilan de competences AI Engineer	56
8.7	Perspectives d'evolution	56
8.7.1	Court terme (phase pilote)	56
8.7.2	Moyen terme	57
8.7.3	Long terme	57

8.8 Conclusion	57
Conclusion generale	58
Bibliographie	59
A Annexes	60
Annexes	60

Table des figures

1.1	Infographie de synthese : du chaos operationnel multi-canal (WhatsApp, Facebook, telephone, comptoir) au workflow supervise AutoFlow. La partie centrale represente l'ossature du systeme, la partie droite le resultat metier attendu (file de validation lisible, brouillons prêts, KPI auditables).	14
4.1	Vue contexte C4. Les acteurs humains (Client, Equipe, Manager) et les systemes externes (LLM optionnel, n8n, WhatsApp Business) sont representees autour du systeme central. Le trait pointille marque les integrations externes non obligatoires.	30
4.2	Vue conteneurs C4. Le frontend communique avec l'API par HTTPS/JSON. L'orchestrateur pilote les agents et journalise dans SQLite/Postgres. Le checkpointeur LangGraph utilise une base dediee pour la reprise apres arret.	31
4.3	Graphe d'orchestration LangGraph. Les losanges representent les gardes conditionnelles (<code>route_after_intake</code> et <code>decide_after_availability</code>). Le noeud <code>human_review</code> en style « gate » materialise le point d'interruption humain. Le graphe est aussi expose par <code>GET /api/graph</code> et affiche dans l'espace admin : le code et le schema <i>ne peuvent pas diverger</i>	32
4.4	Machine a dix etats. Les etats terminaux (<code>confirmed</code> , <code>closed</code>) sont en vert. Les etats d'alerte (<code>needs_human</code> , <code>escalated</code> , <code>stalled</code>) sont en orange. Les transitions inverses sont possibles depuis <code>needs_human</code> vers <code>checked</code> (apres completion) et depuis <code>pending_customer</code> vers <code>confirmed</code> ou <code>closed</code>	33
4.5	Diagramme de sequence du scenario C. Deux <code>interrupt()</code> successifs : le premier pour clarifier les dates, le second pour valider la remise au niveau manager. Chaque etape ecrit un evenement dans la base.	35
4.6	Trois topologies de deploiement. Les secrets (<code>ADMIN_TOKEN</code> , cles LLM) sont exclusivement en variables d'environnement ; le frontend ne contient aucun secret.	36
5.1	Rappel visuel du graphe d'orchestration (cf. chapitre 4). Le noeud <code>human_review</code> est le point de suspension persiste.	42
6.1	Schema de la file a valider. La demande expose son niveau de revue, sa priorite, son motif et les actions disponibles avant toute sortie client.	45
6.2	Schema du suivi explicable : flux d'evenements, chemin execute et regles evaluees. Chaque decision est rattachee a une valeur, un seuil et une branche.	45
6.3	Catalogue des simulations metier. Les cas couvrent les parcours standard, indisponible, ambigu, sensible, reclamation, relance et information.	46

Liste des tableaux

1.1	Les six frictions operationnelles (hypotheses a valider en audit).	15
2.1	Comparaison des cadres d'orchestration multi-agents (septembre 2026). . .	20
2.2	Stack technique d'AutoFlow — decision records synthetises.	22
3.1	Acteurs du systeme AutoFlow et leurs attentes prioritaires.	24
3.2	Exigences fonctionnelles et moyens de verification associes.	25
3.3	Exigences non fonctionnelles et metriques cibles.	26
3.4	Matrice d'alignement entre objectifs pedagogiques et realisations.	27
4.1	Matrice d'escalade appliquee apres verification de disponibilite.	33
4.2	Vue defaillances — comportement du systeme sous stress.	36
6.1	Extrait des routes principales de l'API.	44
7.1	Suite de tests d'integration — couverture des exigences fonctionnelles. . . .	49
7.2	Exactitude par champ sur 45 messages etiquetes (moteur rules , hors ligne). 50	
7.3	Comportement du systeme sous stress — verifie par tests dedies.	51
7.4	Cadre de mesure des indicateurs metier — <i>aucun gain n'est promis avant mesure pilote.</i>	52
7.5	Ce qui rend le projet credible pour ses deux publics.	53

Introduction generale

La dernière decennie a vu le tissu economique marocain se digitaliser a un rythme inegal. Les grandes structures se sont dotées d'ERP, de CRM et de plateformes analytiques ; les tres petites entreprises, majoritaires en volume, continuent de piloter leur relation client au fil de l'eau, entre WhatsApp, Facebook Messenger, appels telephoniques, formulaires web et carnets papier. Cette **fracture d'outillage** n'est pas anecdotique : elle se traduit chaque jour en re-saisies, en delais de reponse allonges, en devis oublies et en clients silencieux qui finissent par acheter ailleurs.

Le present rapport documente un stage realise sous l'intitule *AI Engineer — automatisation des workflows metier*, dont la mission etait de concevoir, realiser et evaluer un prototype de systeme d'automatisation intelligent applicable aux PME de services. Le cas d'application retenu est une **agence de location de voitures a Rabat**, secteur representatif ou la demande arrive massivement par WhatsApp, ou la disponibilite se verifie encore sur un tableur, et ou la moindre remise doit rester une decision humaine.

Le projet AutoFlow prend le contre-pied de la promesse dominante des *agents LLM autonomes*. Plutot qu'un modele libre equipe d'outils qui deciderait a la place de l'equipe, il propose un **workflow supervise** :

- **Un orchestrateur LangGraph** pilote une machine a dix etats et applique une matrice d'escalade explicite.
- **Trois agents specialises** (Intake, Disponibilite, Suivi) accomplissent chacun une responsabilite unique.
- **Un point de controle humain** obligatoire (`interrupt()`) suspend le graphe pour toute decision commerciale.
- **Un journal d'evenements** exhaustif rend chaque decision auditable et chaque indicateur calculable.

Ce parti-pris architectural repose sur trois invariants non negociables : *le systeme n'envoie jamais un message client seul, le systeme n'invente jamais une disponibilite ou un champ non prouve par le message source, et tout est trace*. Ces invariants transforment la question de la confiance — centrale pour tout systeme d'IA en production — en propriete structurelle du code, verifiable par le jury comme par le gerant.

Le manuscrit est organise en **sept chapitres**. Le premier pose le contexte de l'entreprise et la problematique metier. Le deuxieme presente l'etat de l'art 2026 des cadres d'orchestration d'agents et justifie les choix techniques. Le troisieme formalise le cahier des charges. Le quatrieme detaille la conception architecturale selon la methodologie C4 en sept vues, la machine a etats et la matrice d'escalade. Le cinquieme decrit la realisation

des trois agents et de l'orchestrateur. Le sixieme presente l'API FastAPI, l'espace d'administration React et les integrations. Le septieme rend compte de l'evaluation quantitative (tests, exactitude, calibration, robustesse) et cadre la mesure des indicateurs metier en pilote. Une discussion critique, un bilan de competences et les perspectives concluent le rapport.

Note de lecture

Les chiffres cites dans ce rapport sont reproductibles par les commandes `pytest -q` et `python -m eval.eval_intake` dans le depot du projet. Les diagrammes et schemas d'interface sont packages localement dans `rapport_assets/figures/`; leurs sources editables sont dans `rapport_assets/sources/`. Les figures de l'espace admin sont des schemas de navigation et de tracabilite, non des captures d'un jeu de donnees reel.

Chapitre 1

Contexte General et Problematique

Ce premier chapitre etablit le socle du projet AutoFlow. Il decrit l'entreprise d'accueil et son marche, restitue une journee-type d'operations telle qu'observee lors de la phase d'audit, formalise les six frictions structurantes qui ont motive l'intervention, precise les objectifs pedagogiques, techniques et fonctionnels du stage, et presente l'organisation logique du manuscrit.

1.1 Introduction

La conception d'un systeme d'automatisation credible commence par une comprehension fine du contexte metier. Ce chapitre ne cherche pas a affirmer des faits universels sur le marche de la location marocain, mais a documenter avec honnetete ce qui a ete observe, les hypotheses formulees et les objectifs poursuivis. Il pose les fondations sur lesquelles se construiront le cahier des charges, l'architecture et l'evaluation.

1.2 L'entreprise d'accueil et son marche

L'entreprise d'accueil est une **agence de location de voitures de taille moyenne** operant a Rabat, avec trois points de service : Agdal, Hay Riad et l'aeroport Rabat-Sale. Sa flotte comprend une vingtaine de vehicules repartis en quatre categories : citadines (Dacia Sandero et Logan, Renault Clio, Hyundai i10, Kia Picanto, Peugeot 208, Toyota Yaris), berlines (Volkswagen Polo, Hyundai Accent, Toyota Corolla, Skoda Octavia, Mercedes Classe A), SUV (Dacia Duster, Hyundai Tucson, Kia Sportage, Hyundai Creta, Peugeot 3008, Toyota Hilux) et utilitaires (Renault Kangoo, Fiat Doblo). Les tarifs journaliers s'echelonnent de 230 a 800 MAD selon la categorie et la saison.

Le marche marocain de la location courte duree presente plusieurs specificites qui orientent la conception :

- Une **saisonnalite marquee** avec des pics estivaux (juin-aout), autour des ceremonies (mariages) et durant les vacances scolaires.
- Une **multi-canalite extreme** : WhatsApp est le canal dominant (estimation interne 60 %), suivi de Facebook et Instagram (20 %), du telephone (10 %), du formulaire web (5 %) et du comptoir (5 %).
- Un **multilinguisme informel** : les messages sont majoritairement en francais,

souvent en darija (*arabe dialectal marocain*), parfois mélangés dans la même phrase.

- Une **expression des dates variée** : le même intervalle peut s'écrire “du 12 au 15 octobre”, “12/10 au 15/10”, “le week-end prochain” ou “vers la fin du mois”.
- Une **sensibilité au prix** et une pratique répandue de la négociation, notamment sur les longues durées et pour les clients réguliers.

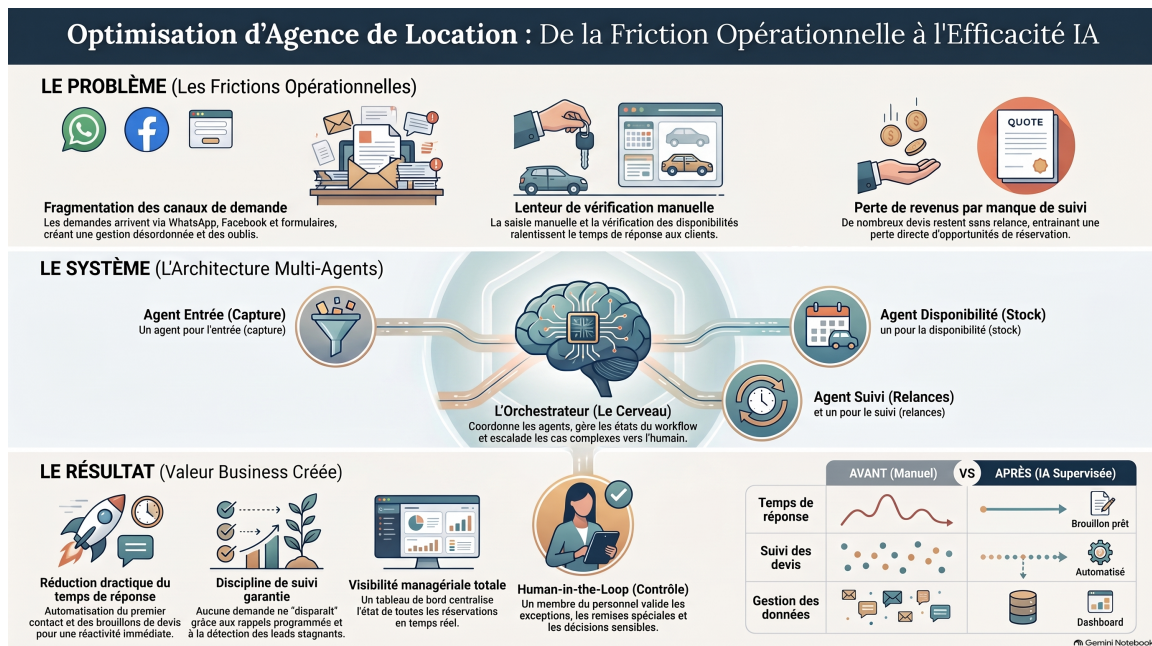


Figure 1.1 – Infographie de synthèse : du chaos opérationnel multi-canal (WhatsApp, Facebook, téléphone, comptoir) au workflow supervisé AutoFlow. La partie centrale représente l'ossature du système, la partie droite le résultat métier attendu (file de validation lisible, brouillons prêts, KPI auditables).

1.3 Observation d'une journée-type

L'audit d'observation, mené sur site pendant deux journées, a permis de reconstruire la séquence typique d'une matinée opérationnelle. Cette narration, qui n'a pas vocation à être exhaustive, illustre le problème mieux qu'une liste théorique :

Une matinée dans l'agence — reconstruction narrative

Une demande arrive sur WhatsApp pendant qu'un client est au comptoir ; un collègue note les dates sur un carnet ; un autre ouvre le tableur des véhicules ; on vérifie si le SUV est revenu de maintenance ; un devis part par retour de message ; le client ne répond pas ; personne n'a le temps de relancer ; trois jours plus tard il a loué ailleurs ; le gérant demande « on en est où ? » et la réponse dépend de la mémoire de chacun.

Cette scene, banale, condense les six frictions structurantes que la section suivante formalise. Elle montre egalement que le probleme n'est *pas* la vitesse individuelle des employes — qui sont competents et efficaces — mais la **coordination** et la **tracabilite** qui manquent des que plusieurs canaux, plusieurs demandes et plusieurs collaborateurs s'entrelacent.

1.4 Les six frictions operationnelles

L'analyse de l'audit permet de synthetiser six frictions structurantes, notees F1 a F6. Ces frictions sont presentees comme des *hypotheses a valider*, jamais comme des faits definitifs : la grille d'audit du kit de prospection permet ulterieurement de les quantifier avec l'agence.

Table 1.1 — Les six frictions operationnelles (hypotheses a valider en audit).

#	Friction observee	Cout probable
F1	Demandes dispersees sur plusieurs canaux	Re-saisie, informations incompletes, oublis silencieux
F2	Disponibilite verifiee a la main dans un tableau	Reponses lentes, risque de double reservation, indisponibilites manquees
F3	Questions repetitives (prix, caution, kilometrage, livraison)	Attention de l'equipe consommee sur du « bruit »
F4	Relances oubliees apres devis	Devis perdus sans que personne ne le voie
F5	Statut des demandes flou (<i>qui gere ? ou en est-on ?</i>)	Coordination par messages et memoire, doublons
F6	Visibilite du gerant limitee	Pilotage a l'instinct, pas de mesure objective

Ces frictions n'apparaissent dans *aucun bilan comptable*. Elles se paient en minutes perdues, en charge mentale et en clients silencieux — trois metriques dont la mesure requiert justement l'outil que ce stage vise a produire.

1.5 Problematique

Problematique centrale du stage

Comment automatiser le traitement des demandes clients d'une PME de services **sans retirer la decision aux humains ni inventer d'information**, et comment *prouver* — a un jury academique comme a un gerant non technique — que le systeme resultant est **fiable, explicable et mesurable** ?

Cette formulation cristallise trois exigences simultanees qui, prises ensemble, excluent plusieurs approches populaires :

1. *Sans retirer la decision aux humains* exclut les agents autonomes qui envoient, remboursent ou refusent sans validation.
2. *Sans inventer d'information* exclut les extractions LLM non ancrees et les generations de disponibilite hallucinees.
3. *Prouver au jury et au gerant* exclut les architectures “boite noire” inexplicables.

1.6 Objectifs du stage

1.6.1 Objectif pedagogique

Le projet vise a maitriser la chaine complete d'un *AI Engineer* moderne : conception d'architecture multi-agents, orchestration de workflows persistants (LangGraph avec `interrupt` / `Command`), ingenierie de prompts structures (*structured output*, *evidence gate*, calibration de confiance), API REST (FastAPI, Pydantic), interface d'operateur (React + TypeScript), tests d'integration (pytest), evaluation (metriques, calibration), deploiement conteneurise (Docker, Vercel/Render). Il exige egalement le developpement de competences transverses : communication metier a un dirigeant non technique, kit de prospection commerciale, conformite reglementaire (loi 09-08 marocaine).

1.6.2 Objectif technique

Livrer un prototype V1 demonstrable :

- **Backend** FastAPI exposant ≥ 20 routes, orchestrateur LangGraph a 7 noeuds, 3 agents Python, persistance SQLite $\rightarrow$ PostgreSQL, checkpointing.
- **Frontend** React 19 + Vite + TypeScript, 10 ecrans admin, formulaire client public.
- **Donnees** : generateur d'agence realiste, 6 semaines d'historique rejouees a travers le vrai graphe.
- **Tests** : ≥ 15 tests d'integration verts, 7 simulations metier jouables.
- **Deploiement** : Docker Compose local, deployable sur Vercel + Render / Railway / Fly, ou tout-Vercel serverless.

1.6.3 Objectif fonctionnel

Ameliorer, pour l'agence d'accueil, la **lisibilite** de sa file de demandes (savoir a tout instant qui gere quoi, dans quel etat, pourquoi), la **securite** de ses reponses client (aucune information inventee, aucun envoi automatique) et la **mesurabilite** de sa performance (KPI calculables depuis un journal, comparaison avant/apres pilote). L'objectif n'est *pas* de reduire la charge de travail commerciale — c'est un effet possible a mesurer en pilote — mais d'**organiser** l'existant.

1.7 Organisation du rapport

Le manuscrit se deroule en sept chapitres logiquement enchaines. Le present chapitre pose le contexte. Le **chapitre 2** presente l’etat de l’art des systemes multi-agents et des cadres d’orchestration en septembre 2026, et justifie les choix techniques par des *decision records*. Le **chapitre 3** formalise le cahier des charges (acteurs, exigences fonctionnelles et non fonctionnelles, perimetre, contraintes). Le **chapitre 4** detaille la conception architecturale selon la methodologie C4 en sept vues, la machine a dix etats et la matrice d’escalade. Le **chapitre 5** decrit la realisation des trois agents et de l’orchestrateur Lang-Graph, avec extraits de code commentes. Le **chapitre 6** presente l’API FastAPI, l’espace d’administration React et les integrations n8n. Le **chapitre 7** rend compte de l’evaluation quantitative (16 tests, exactitude, calibration, robustesse) et cadre la mesure des indicateurs metier. Une conclusion generale, une bibliographie et des annexes cloturent le rapport.

1.8 Conclusion

Ce chapitre a pose le contexte metier et formule la problematique centrale du stage. L’agence de location de voitures de Rabat presente un cas d’usage representatif des PME de services marocaines : multi-canalité, multilinguisme informel, sensibilite au prix, absence d’outillage integre. Les six frictions F1–F6 identifiees en audit motivent une intervention cillee, dont la valeur reside moins dans la reduction d’effectif que dans la mise en ordre de la coordination et la production d’une visibilite auditable. Le chapitre suivant examine l’etat de l’art des solutions techniques susceptibles de repondre a cette problematique et justifie les choix retenus par des *decision records* explicites.

Chapitre 2

Etat de l'Art et Choix Techniques

Ce chapitre positionne AutoFlow dans le paysage technologique de septembre 2026. Il passe en revue les approches classiques d'automatisation (RPA, low-code) et leurs limites face au langage naturel, presente les cadres d'orchestration d'agents fondees sur les LLM (LangGraph, Microsoft Agent Framework, CrewAI, AutoGen), analyse les techniques d'extraction d'information structuree *gardees par la preuve*, discute la place de l'humain dans la boucle (*human-in-the-loop*) et conclut par un tableau de decisions techniques explicites.

2.1 Introduction

Un projet d'IA en 2026 se construit rarement *ex nihilo* : il assemble des briques open source matures dont il faut connaitre les forces, les faiblesses et les tendances d'adoption. Ce chapitre etablit la cartographie technique dans laquelle AutoFlow s'inscrit, puis justifie chaque choix majeur par un *decision record* qui explicite le contexte, l'option retenue, les alternatives ecartees et le cout de retour arriere.

2.2 Automatisation classique : RPA et plateformes low-code

Les plateformes d'**automatisation robotisee des processus (RPA)** — UiPath, Automation Anywhere, Blue Prism — et les plateformes **low-code** — n8n, Make, Zapier, Power Automate — constituent la premiere generation d'outils d'automatisation d'entreprise. Elles excellent dans un scenario bien defini : *relier des applications, transformer des payloads structures, declencher des actions sur des evenements calendaires ou webhooks*. Un devis Salesforce declenche l'envoi d'un email Mailchimp, une commande Shopify cree une ligne dans Google Sheets, un formulaire Typeform poste dans Slack : ces cas sont resolus en minutes.

Leur limite apparait des que **l'entree est du langage naturel** et que **l'etat metier** doit etre tenu et raisonne. n8n ne comprend pas « je voudrais un SUV a partir du week-end prochain, avec livraison a l'aeroport si possible » : il faut lui donner des champs deja extraits. Il ne sait pas non plus qu'une demande « attend le client depuis 48h et il faut la relancer » : il faut lui construire un cron. Ces plateformes restent donc precieuses en **peripherie** d'un systeme, comme colle vers les canaux et couche de notifications, mais elles ne peuvent pas etre le **coeur de decision** d'un workflow qui traite du langage.

2.3 Agents LLM et cadres d'orchestration

2.3.1 L'émergence des cadres orientés graphes d'états

Depuis 2024, une convergence architecturale s'est produite : les cadres d'orchestration d'agents ont abandonné le paradigme du « LLM libre équipé d'outils » pour adopter des **graphes d'états persistants**. La motivation est double : besoin de *reprise après arrêt* (checkpointing), et besoin de *validation humaine* sur les décisions à enjeu.

2.3.2 LangGraph (LangChain)

LangGraph modélise un workflow comme un graphe dont les nœuds sont des fonctions Python et les arêtes sont des décisions conditionnelles. Il offre le **checkpointing** (persistance par `SqliteSaver`, `PostgresSaver`, etc.), l'**interruption** pour validation humaine (`interrupt(payload)`) et la reprise par commande (`Command(resume=decision)` ou `Command(goto=nœud)`). Le choix est justifié ici par ces propriétés vérifiables dans le code — persistance, suspension, reprise et visualisation du graphe — plutôt que par des compteurs publics instables. Sa documentation et son intégration à l'écosystème LangChain (`init_chat_model`, `with_structured_output`) en font un choix adapté à un prototype supervisé.

2.3.3 Microsoft Agent Framework

Le **Microsoft Agent Framework** (2025–2026) propose des graphes de workflows, l'observabilité intégrée (OpenTelemetry, Azure Monitor) et le checkpointing en Python, .NET et Go. Il est particulièrement attractif pour les organisations déjà engagées dans l'écosystème Azure. Sa courbe d'adoption est prometteuse (13,6 k étoiles en un an), mais son intégration à l'infrastructure Azure — Key Vault, Service Bus, Cosmos DB — consommerait le budget-temps d'un pilote de PME sans apporter de valeur incrementale. Il est retenu comme candidat sérieux pour une *version entreprise ultérieure*, hors périmètre du présent stage.

2.3.4 CrewAI et AutoGen

Les cadres **CrewAI** et **AutoGen** privilégient la *conversation entre agents* : plusieurs LLM se parlent pour résoudre une tâche. Ce paradigme excelle pour des tâches ouvertes de recherche ou de création collective, mais s'accommode mal d'une **machine à états explicite** avec transitions légales, ce qui est justement la contrainte imposée par la matrice d'escalade d'AutoFlow.

2.3.5 Standards émergents : MCP et A2A

Deux standards émergent en 2025–2026 : **MCP (Model Context Protocol)**, proposé par Anthropic, qui standardise l'exposition d'outils aux modèles LLM ; et **A2A (Agent-to-Agent)**, qui spécifie la communication entre agents autonomes. AutoFlow reste en **veille** sur ces standards : MCP deviendra pertinent si un LLM avec outils est retenu après le pilote ; A2A est inutile à un seul orchestrateur.

Table 2.1 – Comparaison des cadres d'orchestration multi-agents (septembre 2026).

Critere	LangGraph	MS Agent Framework	CrewAI / Auto-Gen
Paradigme	Graphe d'états + noeuds	Graphe de workflows + observabilité	Conversation multi-agents
HITL natif	✓(interrupt)	✓	Partiel
Checkpointing	✓(SqliteSaver, PostgresSaver)	✓	Non
Adoption 2026	~42k étoiles	~13,6k étoiles	Communautaire
Dependance cloud	Aucune	Azure	Aucune
Cout d'entrée PME	Faible	Élevé (infra Azure)	Faible

2.4 Extraction d'information structurée et confiance

2.4.1 Le problème de l'hallucination

Les LLM produisent des sorties structurées fiables lorsqu'un schéma est imposé (*structured output* via `with_structured_output`, JSON Schema, fonction calling), mais restent **intrinsèquement sujets à l'hallucination** : ils peuvent générer une date qui n'est pas dans le message source, une catégorie de véhicule non demandée, un lieu inventé. Cette propriété rend l'extraction LLM *non deployable seule* dans un contexte commercial où l'agent commercial engage la parole de l'entreprise.

2.4.2 L'ancrage par la preuve (*evidence gate*)

Deux garde-fous ont été retenus. Le premier est l'**ancrage par la preuve** : chaque champ produit par le LLM doit citer un passage *exactement présent* dans le texte source. Techniquement, après extraction, on vérifie que le champ `evidence[k]` est une sous-chaine de la variante normalisée du message ; sinon, le champ est **rejeté** (valeur `null`, confiance = 0). Ce mécanisme rend l'hallucination *structurellement impossible pour les champs*.

2.4.3 La calibration de la confiance

Le second est la **calibration** : un score de confiance n'a de valeur que s'il prédit réellement la justesse. La calibration se mesure : sur les extractions ayant reçu une confiance $\geq 0,8$, quelle proportion est effectivement correcte ? AutoFlow expose `field_confidence[k]`

par champ et une **confidence** globale, comparees a des seuils versionnes dans `rules.json`. La section 7.4 confirme empiriquement que le seuil est bien place.

2.4.4 Inspiration : Jev AI — « probabilites, pas prose »

L'approche de *Jev AI (TypeSafe)* — « des decisions typees avec une probabilite calibree, pas de la prose » — inspire la representation des sorties de l'agent Intake sous forme de **signals** numeriques : `is_complaint: 0.92`, `discount_requested: 0.85`, `dates_ambiguous: 0.55`. L'orchestrateur ne lit *jamais* du texte libre : il compare des nombres a des seuils. Cette discipline supprime la principale source de fragilite des orchestrateurs « conversationnels » : la sensibilite au phrasing des sorties LLM.

2.5 Human-in-the-loop : de la contrainte a la feature

La litterature sur l'automatisation sure converge : les decisions a enjeu — argent, identite, emotion — doivent rester humaines. L'ecole de *safe AI* formalise ce principe sous le nom de **human-in-the-loop (HITL)** : l'humain n'est pas un repli de l'IA faible, mais un **point de controle structurel** qui garantit la responsabilite juridique et commerciale.

Dans AutoFlow, HITL n'est pas un *fallback* d'exception : c'est une **feature centrale**. Chaque demande passe par un `interrupt()` au moins une fois avant d'atteindre le client. Le payload d'interruption — persiste par le checkpointeur LangGraph — contient la raison de la revue, le niveau requis (equipe ou manager), le brouillon prepare et les actions autorisees. L'humain *approuve*, *modifie*, *refuse*, *complete* ou *clot* ; l'orchestrateur reprend via `Command(resume=...)` ou `Command(goto=availability)`.

Le contrat de fiabilite d'AutoFlow

Trois invariants non negociables :

1. Le systeme **n'envoie jamais** un message au client seul (l'envoi est un clic humain).
2. Le systeme **n'invente jamais** une disponibilite (code deterministe) ni un champ (evidence gate).
3. **Tout est trace** : chaque transition ecrit un evenement (acteur, raison, horodatage).

Ces trois invariants sont verifiables par relecture de code et par tests d'integration (chapitre 7).

2.6 Synthèse : le stack technique retenu

Table 2.2 — Stack technique d'AutoFlow — decision records synthetises.

Composant	Techno	Justification courte
Orchestrateur	LangGraph 1.2	HITL + checkpointing natifs, graphe lisible, adoption
Backend API	FastAPI + Pydantic	Schemas partages avec les agents, OpenAPI gratuit
Persistence	SQLAlchemy 2 + SQLite/Postgres	Laptop en pilote, cloud sans redesign
Extraction LLM	LangChain with_structured_output + <i>evidence gate</i>	Optionnel, anti-hallucination structurelle
Frontend	React 19 + Vite + TypeScript	Build statique deployable sur Vercel/-Netlify
Diagramme workflow	Mermaid inline (SVG)	Le graphe et le code ne divergent pas
Notifications	n8n (community)	280+ templates, notifie sans envoyer
Tests	pytest + FastAPI TestClient	16 tests couvrant les scénarios métier
Deploiement	Docker + Vercel/Render/Fly	Cout $\approx$ 0 MAD en pilote

2.7 Modele economique pour l'agence

Le stack a ete choisi non seulement pour ses proprietes techniques, mais aussi pour son **cout d'operation quasi nul**. Un pilote peut tourner sur le laptop du gerant sans installation payante ; une phase distante utilise les offres gratuites de Vercel + Render + Neon. Le LLM est optionnel : en mode `LLM_PROVIDER=none`, le moteur de regles suffit a operer, et le cout d'appel LLM (Anthropic Haiku, OpenAI GPT-4o-mini) plafonne a quelques centimes de MAD par demande. Toutes les licences sont libres (MIT/Apache).

Positionnement commercial : *un outil quasi gratuit a faire tourner, facture sur le temps d'audit et d'adaptation, jamais sur une licence.*

2.8 Conclusion

Ce chapitre a etabli que le paysage 2026 privilegie les **graphes d'etats persistants** sur les agents LLM libres. LangGraph a ete retenu pour sa maturite, son support natif du HITL et son integration a l'ecosysteme LangChain. L'extraction d'information est *gardee par la preuve* et exprimee sous forme de signaux calibres. L'humain dans la boucle est promu au rang de *feature centrale*, non de repli. Le chapitre suivant traduit ces choix en un cahier des charges fonctionnel et non fonctionnel formel.

Chapitre 3

Analyse des Besoins et Cahier des Charges

Ce chapitre transforme les intentions du chapitre 1 et les choix techniques du chapitre 2 en un cahier des charges vérifiable. Il identifie les acteurs du système et leurs attentes, formalise les exigences fonctionnelles (RF1–RF10) et non fonctionnelles, précise le périmètre inclus et exclu, expose les contraintes et hypothèses, et établit la matrice d’alignement avec les objectifs pédagogiques du stage.

3.1 Introduction

Un cahier des charges bien rédigé n’est pas une liste de vœux, c’est un *contrat vérifiable*. Chaque exigence de ce chapitre est associée à un moyen de vérification — test automatisé, code source, mesure — de sorte que la conformité du prototype puisse être objectivement établie au chapitre 7.

3.2 Identification des acteurs

L’analyse fonctionnelle fait émerger quatre acteurs dont les périmètres d’action et les attentes sont distincts. Le tableau 3.1 les synthétise.

Table 3.1 – Acteurs du système AutoFlow et leurs attentes prioritaires.

Acteur	Rôle dans le workflow	Besoins prioritaires
Client	Émetteur des demandes, en langage naturel	Réponse rapide, précise, propositions d’alternatives
Équipe commerciale	Validation, envoi, modification des brouillons, saisie des réponses client	File claire, brouillons pertinents, raison lisible
Manager	Décisions sensibles : remises, réclamations, longues durées, valeurs élevées, refus	Vue synthétique, matrice d’escalade explicite, traçabilité
Système	Structure, vérifie, prépare, route, journalise, rappelle	Règles versionnées, invariants respectés, aucun envoi seul

Le **client** n’interagit *jamaïs* directement avec le système : il écrit sur WhatsApp, Facebook ou remplit un formulaire ; c’est l’équipe qui manipule l’interface. Cette séparation garantit que la relation commerciale reste humaine, même si sa préparation est assistée.

3.3 Exigences fonctionnelles (RF1–RF10)

Table 3.2 – Exigences fonctionnelles et moyens de verification associes.

ID	Exigence	Verification
RF1	Toute demande recue obtient un identifiant, un etat initial et un evenement journalisation	<code>test_scenario_a_standard_quote</code>
RF2	Le systeme ne declare jamais un vehicule disponible sans verification deterministe	<code>availability.py</code> , <code>test_scenario_b</code>
RF3	Un champ extrait par LLM sans preuve verbatim est rejete (<i>evidence gate</i>)	<code>extract_llm</code> + <code>test_llm_gate</code>
RF4	Toute reclamation est escaladee au manager sans brouillon automatique	<code>test_scenario_d_complaint</code>
RF5	Remise demandee, location >14 j, valeur >6000 MAD → manager	<code>decide_after_availability</code>
RF6	Aucun message n'est envoye sans clic humain	<code>finalize_node</code> uniquement via <code>Command(resume)</code>
RF7	Une relance est planifiee a l'envoi ; au-dela de la politique, <i>stalled</i> → humain	<code>test_follow_up_engine</code>
RF8	Chaque transition est journalisee avec acteur, raison, horodatage	<code>store.transition</code>
RF9	Les KPI sont calcules depuis le journal, jamais stockes	<code>orchestrator.kpis</code>
RF10	L'espace admin exige un token ; le formulaire public n'en exige pas	<code>test_auth_required</code>

Ces dix exigences couvrent l'integralite du perimetre V1. Chacune est **tracee a un moyen de verification** : test automatisé, module de code, ou combinaison des deux. La matrice de couverture est presentee au chapitre 7.

3.4 Exigences non fonctionnelles

Table 3.3 – Exigences non fonctionnelles et metriques cibles.

Critere	Exigence	Metrique cible
Autonomie	Fonctionnement hors ligne complet (LLM_PROVIDER=none)	La demo ne depend d’aucune API externe
Explicabilite	Chaque routage doit etre justifiable par une regle et une valeur mesuree	Panneau <i>Suivi en direct</i> avec chemin allume
Reprise	L’etat du workflow survit a un redemarrage	Checkpointing SQLite, <code>pending_interrupt</code> conserve
Confidentialite	Telephones masques, donnees fictives ; seul le texte du message vers un LLM (si active)	Conforme au principe de minimisation
Reglementation	Loi 09-08 (protection des donnees personnelles au Maroc)	A cadrer avec l’agence avant tout usage de donnees reelles
Simplicité	Code metier concis, lisible en journee	< 1500 lignes de Python metier
Deployabilite	Deployable sur laptop, cloud gere ou serverless	Docker Compose, Vercel/Render, Vercel Python
Cout	Infrastructure quasi nulle pour l’agence	≈ 0 MAD en pilote local, ~5-7 \$/mois cloud

3.5 Perimetre inclus et exclu

3.5.1 Fonctionnalites incluses (V1 demonstrable)

Le perimetre V1 comprend : saisie manuelle et formulaire public /demande et webhook n8n ; les trois agents (Intake, Disponibilite, Suivi) ; l’orchestrateur LangGraph ; l’espace d’administration en dix ecrans (tableau de bord, file a valider, suivi en direct, simulations, demandes, fiche demande, nouvelle demande, flotte & regles, architecture, mode presentation) ; le jeu de donnees realiste (24 vehicules, 80 reservations, 40 clients, 45 demandes sur 6 semaines rejouees) ; les sept simulations metier jouables ; le deployment Vercel/Netlify + Docker ou Vercel Python.

3.5.2 Fonctionnalites exclues (*audit-gated*)

Sont **explicitement exclues** du perimetre V1 : le paiement en ligne, la tarification dynamique, l’envoi automatique de messages au client, le connecteur WhatsApp Business ou Instagram *en direct* (n8n reste en peripherie), l’integration CRM/ERP, la maintenance predictive, la voix. Chaque exclusion est *audit-gated* : elle n’entre au perimetre que si l’audit sur site prouve son besoin operationnel.

3.6 Contraintes et hypotheses

- **Environnement de developpement** : Windows 11 avec Python 3.12+, Node 20+, PowerShell. La compatibilite Linux/mac est assuree par le `run.sh` et Docker.
- **Donnees d'entrainement** : *aucune donnee reelle client n'est utilisee*. Le pilote fonctionne sur un jeu synthetique realiste genere de facon deterministe.
- **Compatibilite navigateur** : Chrome, Firefox et Safari recents (ES2020+).
- **Absence de secret dans le frontend** : le token admin est saisi a la connexion et stocke uniquement en `localStorage`.
- **Loi 09-08** : le traitement de donnees personnelles reelles requiert un cadrage formel avec l'agence avant tout deploiement.

3.7 Alignement avec les objectifs pedagogiques

Table 3.4 – Matrice d'alignement entre objectifs pedagogiques et realisations.

Objectif pedagogique	Realisation dans AutoFlow	Etat
Orchestration multi-agents	Orchestrateur LangGraph a 7 noeuds, 3 agents	Realise
Human-in-the-loop persistant	<code>interrupt()</code> + <code>Command(resume/goto)</code> + <code>Sqlite-Saver</code>	Realise
Ingenierie de prompts structures	Sortie typee + <i>evidence gate</i> + signaux calibres	Realise
API REST	FastAPI, 24 routes, OpenAPI genere	Realise
Interface d'operateur	React 19 + Vite + TypeScript, 10 ecrans	Realise
Tests d'integration	16 tests pytest verts (8 scenarios + 7 simulations + catalogue)	Realise
Evaluation quantitative	Rapport d'exactitude, calibration, filet de securite	Realise
Communication metier	Kit de prospection complet (11 documents, visuels PDF)	Realise
Documentation d'architecture	7 vues C4, decision records, 4 diagrammes exportables	Realise

Le cahier des charges est un contrat testable

Contrairement a un cahier des charges narratif, les 10 exigences fonctionnelles et les 8 exigences non fonctionnelles d'AutoFlow sont *tracees a du code executable*. Le chapitre 7 (Evaluation) exhibera la matrice de couverture qui associe chaque exigence a son test.

3.8 Conclusion

Le cahier des charges est formalise, les acteurs identifies, le perimetre precise, les contraintes explicitees. Le contrat V1 tient en dix exigences fonctionnelles et huit exigences non fonctionnelles, chacune verifiable par un moyen precis. Le chapitre suivant traduit ce contrat en une architecture logicielle detaillee, presentee selon la methodologie C4 en sept vues complementaires.

Chapitre 4

Conception Architecturale

Ce chapitre presente la conception architecturale d'AutoFlow selon la methodologie C4 (Context, Containers, Components, Code) enrichie de trois vues complementaires : flux de donnees, sequence et defaillances. Il detaille le graphe d'orchestration LangGraph, la machine a dix etats, la matrice d'escalade et le modele de donnees relationnel. La convention adoptee est stricte : trait plein pour un *FACT* verifie dans le code, trait pointille pour une *HYPOTHESIS* a valider avec l'agence.

4.1 Introduction

L'architecture d'AutoFlow a ete concue selon un principe directeur unique : **la simplicité d'abord**. Un workflow orchestre avec un orchestrateur, trois agents et six outils metier est plus *credible*, plus *testable* et plus *demontrable* que « cinq agents pour faire moderne ». Ce chapitre expose successivement les sept vues qui, ensemble, decrivent le systeme sans jamais en surcharger un seul schema.

4.2 Vue contexte — qui parle au systeme ?

La vue contexte identifie les acteurs humains et les systemes externes qui interagissent avec AutoFlow. La **frontiere-cle**, verifiee dans le code, est qu'AutoFlow *ne communique jamais directement avec le client* : il produit des brouillons qu'une personne clique pour envoyer.

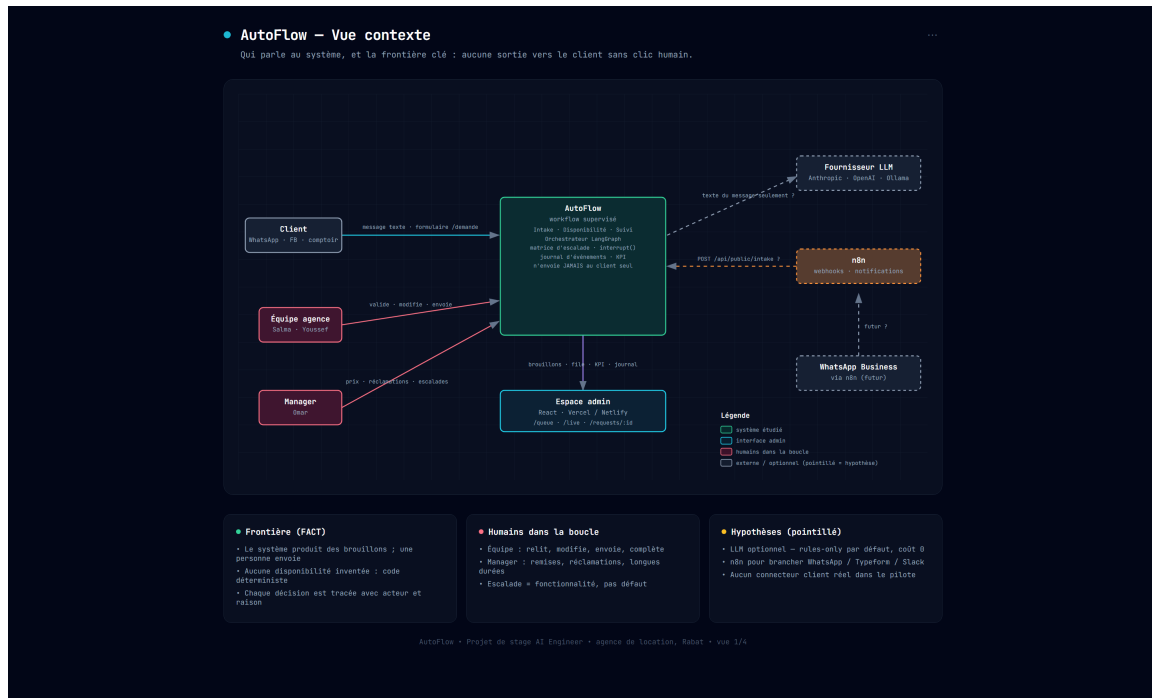


Figure 4.1 — Vue contextuelle C4. Les acteurs humains (Client, Equipe, Manager) et les systèmes externes (LLM optionnel, n8n, WhatsApp Business) sont représentés autour du système central. Le trait pointillé marque les intégrations externes non obligatoires.

4.3 Vue conteneurs — de quoi est fait le système ?

La vue conteneurs décompose AutoFlow en *unités déployables* : un frontend statique React, un backend FastAPI, l'orchestrateur LangGraph, les trois agents, une base relationnelle et un checkpoint. Le frontend est hébergé sur Vercel ou Netlify (build statique, aucun secret), le backend sur Docker (Render / Railway / Fly) ou en serverless Vercel Python.

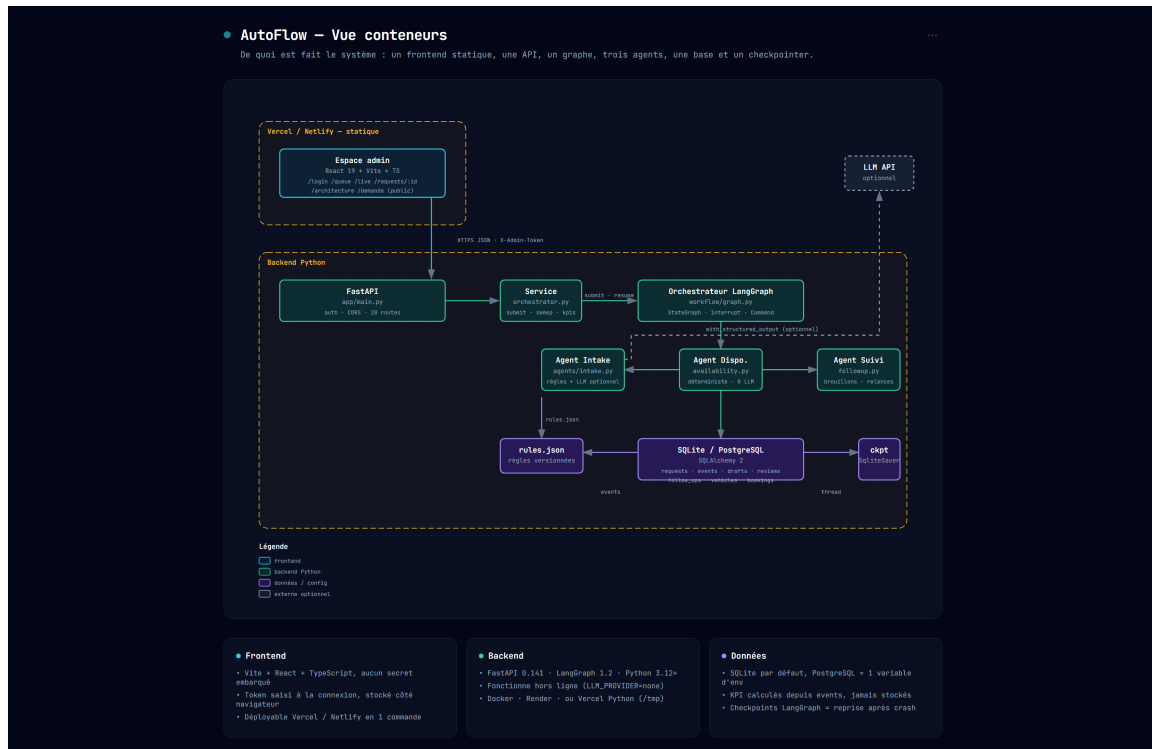


Figure 4.2 — Vue conteneurs C4. Le frontend communique avec l'API par HTTPS/JSON. L'orchestrateur pilote les agents et journalise dans SQLite/Postgres. Le checkpoint LangGraph utilise une base dédiée pour la reprise après arrêt.

4.4 Vue composants — le graphe LangGraph

Le graphe d'orchestration compte **sept noeuds** et **deux gardes conditionnelles**. Le flux nominal est : `START` → `intake` → `route_after_intake` → {`escalate` | `clarify` | `availability`} → `decide_after_availability` → {`sensitive` | `draft`} → `human_review` (interrupt) → `finalize` → `END`. Le noeud `human_review` peut renvoyer vers `availability` par `Command(goto=availability)` après completion des champs manquants par l'humain.

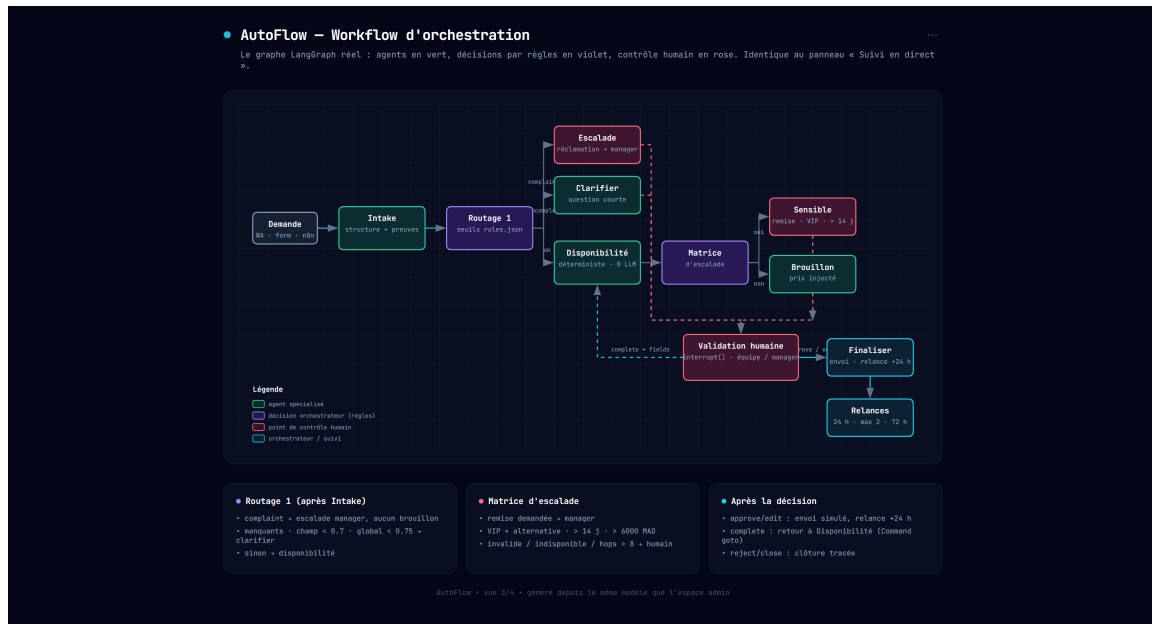


Figure 4.3 – Graphe d’orchestration LangGraph. Les losanges représentent les gardes conditionnelles (`route_after_intake` et `decide_after_availability`). Le noeud `human_review` en style « gate » materialise le point d’interruption humain. Le graphe est aussi expose par `GET /api/graph` et affiche dans l’espace admin : le code et le schema *ne peuvent pas diverger*.

4.5 Machine a dix etats et transitions legales

La machine a etats formalise le cycle de vie d’une demande. Chaque transition passe par `store.transition()` qui refuse un mouvement illegal (leve `IllegalTransition`, HTTP 409) et ecrit un evenement dans la table `events`.

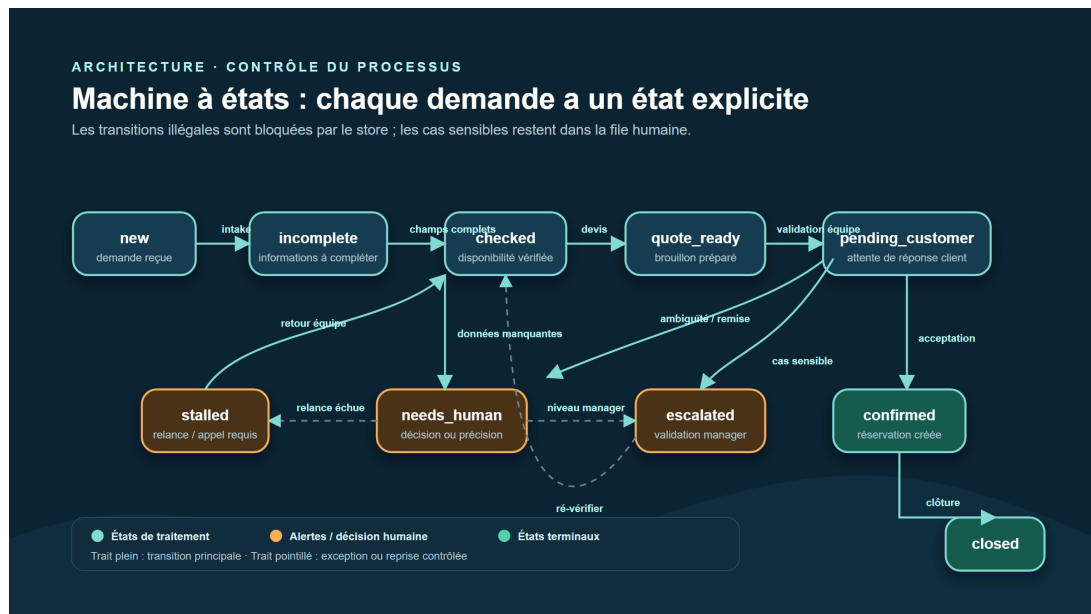


Figure 4.4 – Machine à dix états. Les états terminaux (`confirmed`, `closed`) sont en vert. Les états d’alerte (`needs_human`, `escalated`, `stalled`) sont en orange. Les transitions inverses sont possibles depuis `needs_human` vers `checked` (après completion) et depuis `pending_customer` vers `confirmed` ou `closed`.

Les dix états sont : `new` (demande reçue), `incomplete` (champs manquants), `checked` (Intake termine), `quote_ready` (brouillon prêt), `pending_customer` (envoyé au client, en attente de réponse), `stalled` (client silencieux depuis 72 h), `needs_human` (revue équipe), `escalated` (revue manager), `confirmed` (réservation créée), `closed` (dossier clos).

4.6 Matrice d’escalade

La matrice d’escalade formalise la question centrale : **qui doit décider quoi, et pourquoi** ? Elle est évaluée par la garde `decide_after_availability` après vérification de la disponibilité.

Table 4.1 – Matrice d’escalade appliquée après vérification de disponibilité.

Declencheur	Niveau	Etat cible
Reclamation	Manager	<code>escalated</code>
Champs manquants ou confiance globale < 0,75	Equipe	<code>needs_human</code>
Confiance de champ critique < 0,7	Equipe	<code>needs_human</code>
Remise demandee	Manager	<code>escalated</code>
Client regulier avec alternative proposee	Equipe	<code>needs_human</code>
Duree > 14 jours	Manager	<code>escalated</code>
Valeur > 6000 MAD	Manager	<code>escalated</code>
Statut invalide ou indisponible	Equipe	<code>needs_human</code>
Plus de 8 sauts (<i>anti-boucle</i>)	Equipe	<code>needs_human</code>

Cette matrice est *explicite*, *versionnee* (dans `rules.json`) et *auditable*. C’est elle qui

donne au gerant la certitude qu’aucune remise, aucune reclamation, aucune reservation exceptionnelle ne sera prise sans son accord.

4.7 Vue flux de donnees

Le trajet complet d’un message brut a travers le pipeline est le suivant :

Listing 4.1 – Trajet d’un message brut a travers le pipeline supervise.

```
Message brut (str, francais / darija)
-> Intake      : normalisation NFD + regex dates FR + mots-cles
                + [LLM structured output + preuve verbatim]
-> StructuredRequest {intent, dates ISO, categorie, boite, lieux,
                    flags[], field_confidence{}, confidence,
                    missing[], evidence{}, signals{}}
-> Orchestrateur : comparaison signaux vs rules.json -> route
-> Disponibilite : SELECT vehicles, bookings -> chevauchement
                + tampon 4 h + maintenance + replis
                -> AvailabilityResult {status, options[],
                    alternatives[], reasons[]}
-> Matrice d’escalade -> Review {level, priority, reason}
-> Suivi        : template FR + injection prix depuis la table
                -> FollowUpOutput {draft_kind, draft_fr, action}
-> interrupt()  : payload persiste (checkpoint) ;
                DB : Request.state, Draft, Review, Event
-> Decision humaine (JSON) -> Command(resume) -> finalize
                Draft.sent_at, FollowUp.due_at = now + 24 h
-> Sweep        : FollowUp due -> brouillon relance ;
                > 72 h -> stalled -> needs_human
-> KPI = agregats sur events/drafts/follow_ups (jamais stockes)
```

4.8 Vue sequence — scenario d’ambiguite avec remise

Le diagramme de sequence du scenario C (dates ambiguës + remise demandée) illustre l’aller-retour humain-machine complet : *ambiguite detectee* → *needs_human* → completion manuelle → retour a la disponibilite via `Command(goto)` → matrice detecte la remise → *escalated manager* → approbation avec note → envoi.

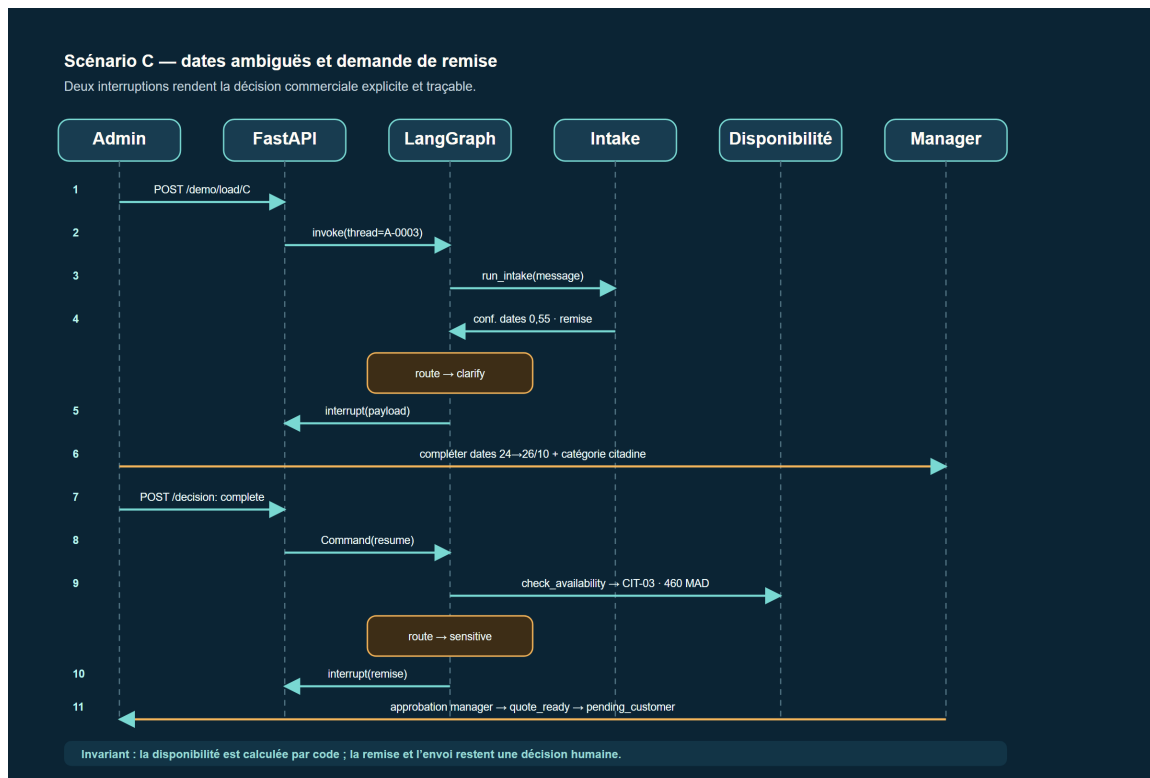


Figure 4.5 — Diagramme de séquence du scénario C. Deux `interrupt()` successifs : le premier pour clarifier les dates, le second pour valider la remise au niveau manager. Chaque étape écrit un événement dans la base.

4.9 Vue déploiement

Trois topologies de déploiement sont supportées : (1) laptop hors ligne pour la demo agence ; (2) frontend Vercel/Netlify + backend Docker sur Render/Railway/Fly avec volume persistant ; (3) tout-Vercel serverless avec base Postgres externalisée (Neon ou Supabase).

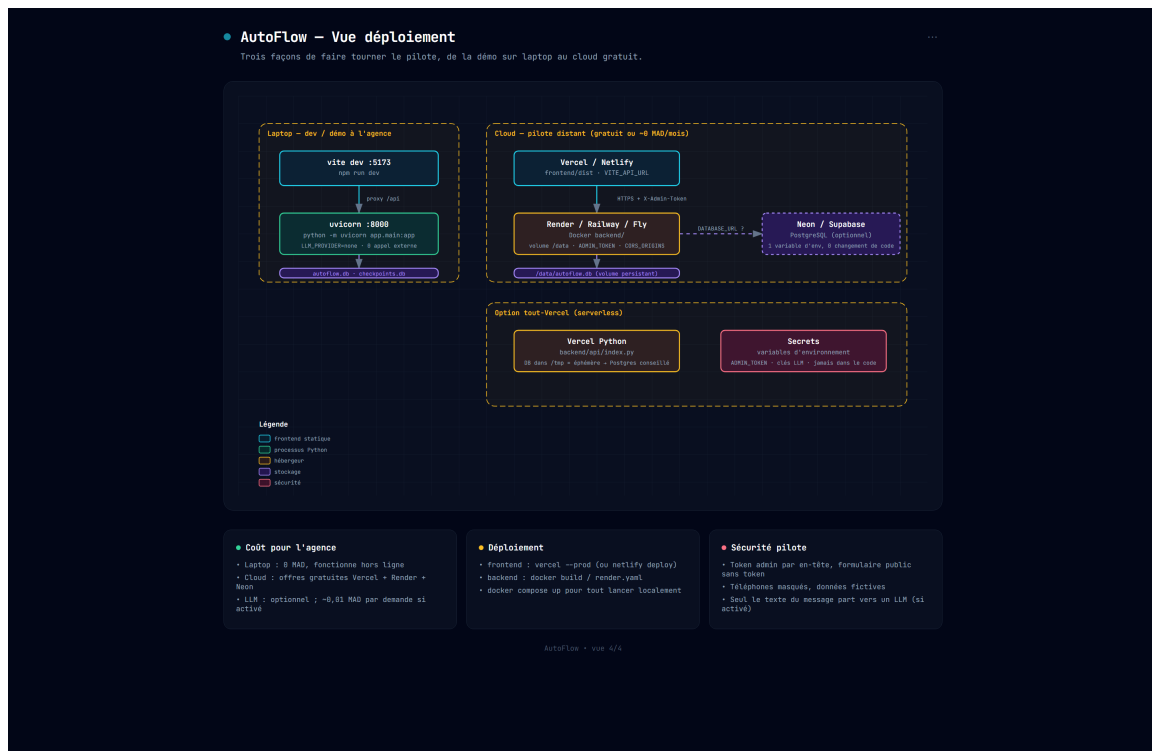


Figure 4.6 – Trois topologies de déploiement. Les secrets (ADMIN_TOKEN, clés LLM) sont exclusivement en variables d’environnement ; le frontend ne contient aucun secret.

4.10 Vue defaillances

Table 4.2 – Vue defaillances — comportement du systeme sous stress.

Defaillance	Comportement	Chemin de secours
LLM indisponible / cle invalide	Flag <code>llm_failed</code> , confiance $\times 0,9$	Regles seules (toujours calculees)
LLM invente un champ	Champ rejete (<code>null</code>), confiance = 0	Question de clarification
Dates ambiguës (< week-end prochain »)	<code>needs_human</code> + brouillon de question	Equipe complete → <code>Command(goto)</code>
Crash pendant le workflow	Checkpoint LangGraph par noeud	Reprise au dernier noeud persiste
Boucle agents (hops > 8)	Route <code>sensitive</code> → humain	—
Transition d'etat illegale	<code>IllegalTransition</code> → HTTP 409	Rien n'est ecrit
Trop de relances (<code>max_reminders</code> = 2)	<code>stalled</code> → appel recom- mande	—

4.11 Conclusion

L’architecture d’AutoFlow est désormais entierement specifiée : sept vues C4 complementaires, une machine a dix etats, une matrice d’escalade explicite, un modele de

donnees derive du journal d'evenements. Cette architecture respecte les trois invariants (aucun envoi seul, aucune information inventee, tout trace) et satisfait toutes les exigences du chapitre 3. Le chapitre suivant en decrit la realisation, agent par agent.

Chapitre 5

Realisation : Agents et Orchestrateur

Ce chapitre decrit la realisation logicielle du coeur du systeme : les trois agents specialises (Intake, Disponibilite, Suivi) et l'orchestrateur LangGraph qui les coordonne. Il commente les choix d'implementation, expose les extraits de code determinants, decrit le mecanisme du checkpointer et explique comment la validation humaine est materialisee par `interrupt()` et `Command`.

5.1 Introduction

La realisation du coeur backend represente environ 1400 lignes de Python metier, reparties en modules courts a responsabilite unique. Le principe « une fonction, une responsabilite » est applique strictement : chaque agent est independamment testable et remplaçable. Ce chapitre parcourt les composants dans l'ordre de leur execution par l'orchestrateur.

5.2 Organisation du code

Listing 5.1 – Arborescence du backend AutoFlow.

```
backend/  
  app/  
    main.py          # FastAPI, 24 routes, auth token  
    db.py            # SQLAlchemy 2.0, TRANSITIONS legales  
    clock.py         # Horloge demo (avance controlee du temps)  
    rules.py         # Chargement de rules.json  
    agents/  
      intake.py      # Comprendre : regles + LLM optionnel  
      availability.py # Verifier : deterministe, aucun LLM  
      followup.py    # Preparer : brouillons + relances  
    workflow/  
      graph.py       # Orchestrateur LangGraph (StateGraph)  
      orchestrator.py # submit, resume, sweep, kpis, seed  
      store.py       # transition, log, get_events, drafts  
      explain.py     # Reconstruction du raisonnement  
      state.py       # WorkflowState (TypedDict)  
      simulations.py # 7 cas metier, catalogue + runner  
  data/  
    generate_dataset.py, fleet.csv, bookings.csv,
```

```
customers.csv, history.json, scenarios.json, rules.json
tests/          # 16 tests (8 e2e + 7 sim + auth)
eval/           # eval_intake.py, report.md
```

5.3 Agent Intake — comprendre le message

5.3.1 Le moteur de regles (toujours execute)

L'agent Intake est double : un moteur de **regles** (regex, grammaire de dates FR/darija, lexiques) est toujours execute, et un moteur **LLM optionnel** peut le completer. Cette double execution permet a AutoFlow de tourner hors ligne et rend le LLM non critique.

Listing 5.2 – Extrait de agents/intake.py — la boucle de fusion des deux moteurs.

```
def run_intake(message: str) -> StructuredRequest:
    normalised = normalise_nfd(message)
    rules_out = extract_rules(normalised)      # deterministe
    if not llm_enabled():
        return _finalise(rules_out, engine="rules")
    try:
        llm_out = extract_llm(normalised)      # avec evidence gate
    except Exception as exc:
        return _finalise(rules_out, engine="rules",
                          flags=[f"llm_failed:{type(exc).__name__}"])
    fused = fuse(rules_out, llm_out)           # accord -> 0.95 ;
    return _finalise(fused, engine="rules+llm") # sinon LLM avec preuve
```

5.3.2 L'evidence gate

Le mecanisme central de l'agent Intake est l'*evidence gate*. Apres extraction par le LLM (via `with_structured_output`), chaque champ est verifie : le passage justificatif (`evidence[k]`) doit etre une sous-chaine exacte du message normalise. Sinon, le champ est **rejete**.

Anti-hallucination structurelle

Si le LLM propose `return_date = "2026-10-27"` et `evidence["return_date"] = "le 27 octobre au soir"` mais que cette chaine n'est *pas* presente dans le message brut, le champ est **rejete**. La demande passe alors en clarification humaine. Aucune date fantome ne peut sortir de l'agent.

5.3.3 Les signaux typés

Enfin, l'agent Intake expose des **signals** : des probabilites calibrees sur des questions oui/non binaires, que l'orchestrateur compare a des seuils. Cette representation *nume-*

rique des decisions — inspiree de Jev AI — supprime la principale source de fragilite des orchestrateurs « conversationnels ».

5.4 Agent Disponibilite — verifier sans LLM

L'agent Disponibilite est **intentionnellement deterministe** : aucun appel LLM. Il resoud le probleme du chevauchement des reservations avec un tampon de 4 heures (nettoyage + reprise), filtre les vehicules en maintenance, propose des *replis* par categorie (`category_fallbacks`) et des *decalages* de dates (`shift_days_allowed`, ± 3 jours). Chaque option ecartee porte une **raison lisible**.

L'argument n°1 de confiance pour le gerant

L'agent Disponibilite ne peut *pas* inventer un vehicule libre : sa logique est du SQL et de la comparaison de dates. Cet argument est central dans la vente au gerant : « c'est du code, pas une IA. Si le vehicule est reserve, le systeme le dit ; il ne peut pas se tromper. »

5.5 Agent Suivi — preparer sans envoyer

L'agent Suivi produit trois types de brouillons FR : *devis* (avec prix injecte depuis la table `vehicles`), *alternative* (quand une option a ete substituee) et *clarification* (quand une question doit etre posee). Il applique une *politique de relance* : la premiere relance est planifiee a +24 h de l'envoi initial, un maximum de deux relances est autorise, et un lead sans reponse depuis 72 h passe en `stalled` avec recommandation d'appel.

Listing 5.3 – Extrait de `agents/followup.py` — template de devis avec prix injecte.

```
def draft_quote(structured: StructuredRequest, avail: AvailabilityResult) -> str:
    option = avail.options[0] # meilleure option classee
    vehicle = fleet.get(option.vehicle_id)
    total = option.days * vehicle.price_per_day
    return TEMPLATES["devis"].format(
        client_first_name=structured.client.first_name or "",
        vehicle_label=f"{vehicle.brand} {vehicle.model}",
        days=option.days,
        price_per_day=vehicle.price_per_day, # jamais genere
        total=total,
        pickup_site=option.pickup_site,
    )
```

5.6 Orchestrateur LangGraph — coordonner et suspendre

5.6.1 Structure du graphe

L'orchestrateur est un StateGraph LangGraph avec sept noeuds (intake, escalate, clarify, availability, sensitive, draft, human_review, finalize) et deux gardes conditionnelles.

Listing 5.4 – Extrait de workflow/graph.py — garde de routage apres Intake.

```
def route_after_intake(state: WorkflowState) -> str:
    s = state["structured"]
    if s.signals.get("is_complaint", 0.0) >= rules["complaint_threshold"]:
        return "escalate"
    if s.missing or s.confidence < rules["confidence_threshold"]:
        return "clarify"
    critical = min(s.field_confidence.get(f, 1.0)
                  for f in rules["critical_fields"])
    if critical < rules["critical_field_threshold"]:
        return "clarify"
    return "availability"
```

5.6.2 Le point d'interruption humain

Le noeud human_review est un interrupt() reel : l'execution est suspendue, le payload persiste par le checkpointer, et l'API expose cet etat comme pending_interrupt. La reprise se fait par Command(resume=decision) pour approuver/modifier/refuser/cloturer, ou Command(goto="availability") apres completion des champs manquants.

5.6.3 Le checkpointer SQLite

Le SqliteSaver de LangGraph persiste l'etat du graphe (variables, position dans le graphe, payload d'interruption) apres chaque noeud dans une base dediee (checkpoints.db). Consequence directe : **une validation en attente survit a un redemarrage** du serveur. Un test dedie (test_kpis_and_graph) verifie cette propriete.

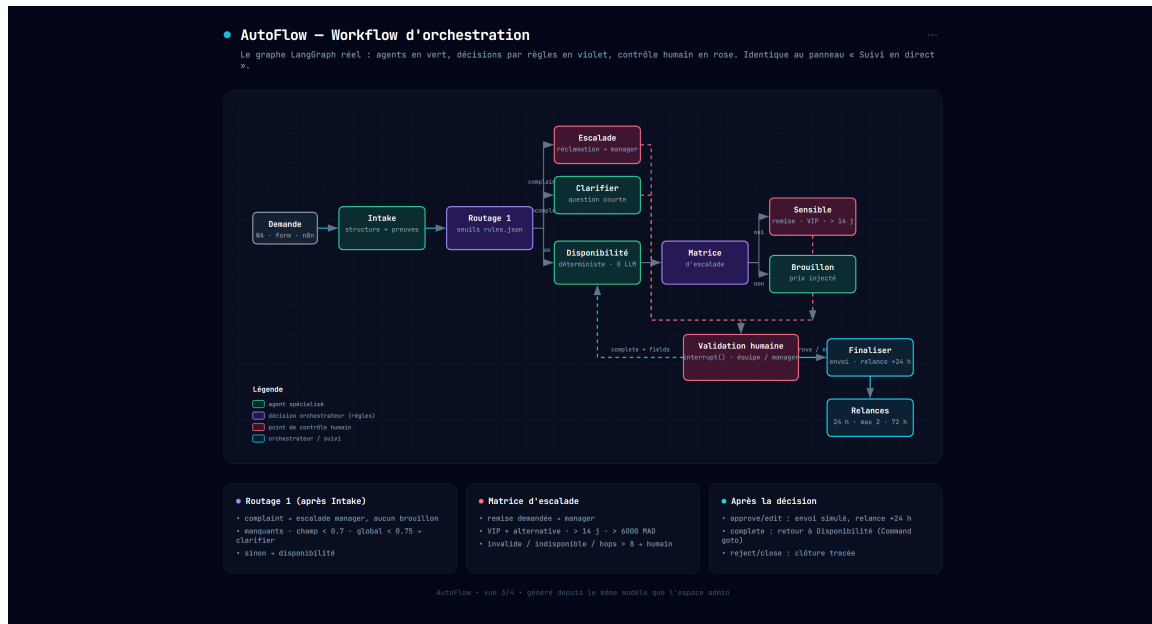


Figure 5.1 – Rappel visuel du graphe d’orchestration (cf. chapitre 4). Le noeud `human_review` est le point de suspension persiste.

5.7 Difficultes rencontrées et résolutions

- **Dates françaises ambiguës** : la forme « du 27 aout au 17 septembre » prenait le second mois pour les deux dates. Correction : ajout d’une capture optionnelle du premier mois dans le regex. Detection : évaluation quantitative de l’agent Intake.
- **Horloge de lead sans réponse** : elle repartait à chaque relance envoyée, rendant `stalled` inatteignable après la deuxième relance. Correction : prendre pour référence la *première réponse envoyée*. Detection : simulation « lead silencieux ».
- **Priorité d’intention** : « je passerai réserver plus tard » faisait primer *reservation* sur *information*. Correction : règle ajoutée (question sans date + mots d’information → `info`). Detection : simulation « question d’information ».
- **Fichiers SQLite verrouillés sous Windows** dans les tests : libération explicite de l’engine et du cache du graphe entre chaque test.

5.8 Conclusion

Les trois agents et l’orchestrateur constituent le cœur exécutable d’AutoFlow. Leur implementation respecte strictement le principe de responsabilité unique, l’invariant « aucune information inventée » (via l’evidence gate et le déterminisme de la disponibilité) et l’invariant « aucun envoi seul » (via `interrupt()`). Le chapitre suivant présente les couches externes : API REST, espace d’administration React et intégrations n8n.

Chapitre 6

Realisation : API, Espace Admin, Integrations

Ce chapitre couvre les couches en peripherie du coeur decrit au chapitre 5 : l'API REST FastAPI qui expose l'orchestrateur, l'espace d'administration React qui materialise la validation humaine, les integrations n8n en peripherie, le generateur de donnees realistes et le catalogue de sept simulations metier.

6.1 Introduction

Un moteur de workflow, aussi bien concu soit-il, reste invisible sans les surfaces qui l'exposent. Ce chapitre presente les quatre surfaces qui rendent AutoFlow utilisable et demonstrable : l'API JSON, l'espace admin, les integrations peripheriques et le systeme de donnees realistes.

6.2 API REST FastAPI

L'API expose **24 routes** organisees en huit familles : `/api/health`, `/api/public/*` (formulaire client, ingestion webhook n8n), `/api/requests/*` (CRUD demandes, decision humaine), `/api/trace/*` (raisonnement, journal), `/api/live/*` (flux d'evenements), `/api/reviews/*` (file), `/api/fleet/*` et `/api/rules/*` (parametres), `/api/kpis`, `/api/events`, `/api/graph`, `/api/simulations/*`, `/api/demo/*` (scenarios, horloge, reset).

Table 6.1 – Extrait des routes principales de l'API.

Methode	Route	Description
POST	/api/public/intake	Ingestion d'un message client (formulaire ou webhook n8n)
POST	/api/requests/{id}/decision	Decision humaine (approve, edit, reject, close, complete)
GET	/api/reviews	File des demandes a valider (equipe / manager)
GET	/api/trace/{id}	Reconstruction du raisonnement d'une demande
GET	/api/live/events	Flux d'evenements en temps quasi reel
GET	/api/graph	Diagramme Mermaid du graphe LangGraph
GET	/api/kpis	Compteurs agreges depuis le journal
POST	/api/demo/load/{scenario}	Charge un scenario de demo (A-E)

L'authentification repose sur un **token unique** passe par en-tete **X-Admin-Token**. Le formulaire public **/demande** ne requiert pas ce token (uniquement en lecture). Le CORS est restreint a l'URL du frontend en production.

6.3 Espace d'administration React

6.3.1 Vue d'ensemble

L'espace admin est une SPA React 19 + Vite + TypeScript, construite en 10 ecrans, sans framework UI kit : uniquement du CSS custom aux couleurs de la charte (navy #0B1F3A / teal #0E9C99 / creme #F7F4EC), avec Manrope comme police, icones SVG, transitions courtes, anneaux de focus, respect de **prefers-reduced-motion**.

6.3.2 La file a valider

L'ecran central est la **file a valider**. Chaque ligne montre l'identifiant, le niveau requis (badge equipe ou manager), la priorite, une raison lisible (« Remise demandee — validation manager »), le brouillon prepare et trois boutons : *Approuver & envoyer*, *Modifier*, *Refuser*.

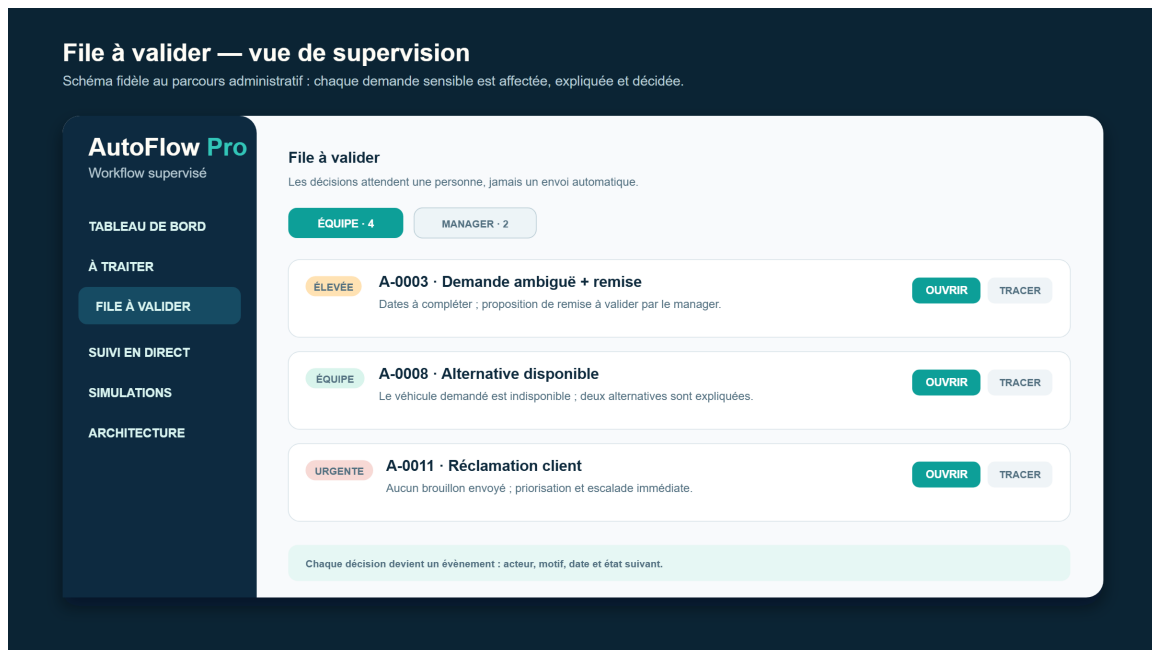


Figure 6.1 — Schema de la file à valider. La demande expose son niveau de revue, sa priorité, son motif et les actions disponibles avant toute sortie client.

6.3.3 Le suivi en direct et le raisonnement

L'écran **Suivi en direct** affiche un flux d'événements en temps quasi réel (rafraîchissement toutes les 4 s) et, pour toute demande sélectionnée, deux panneaux critiques : (1) le graphe du workflow avec le *chemin réellement suivi allumé* en teal ; (2) un tableau listant chaque règle évaluée (nom, valeur mesurée, seuil, branche déclenchée).

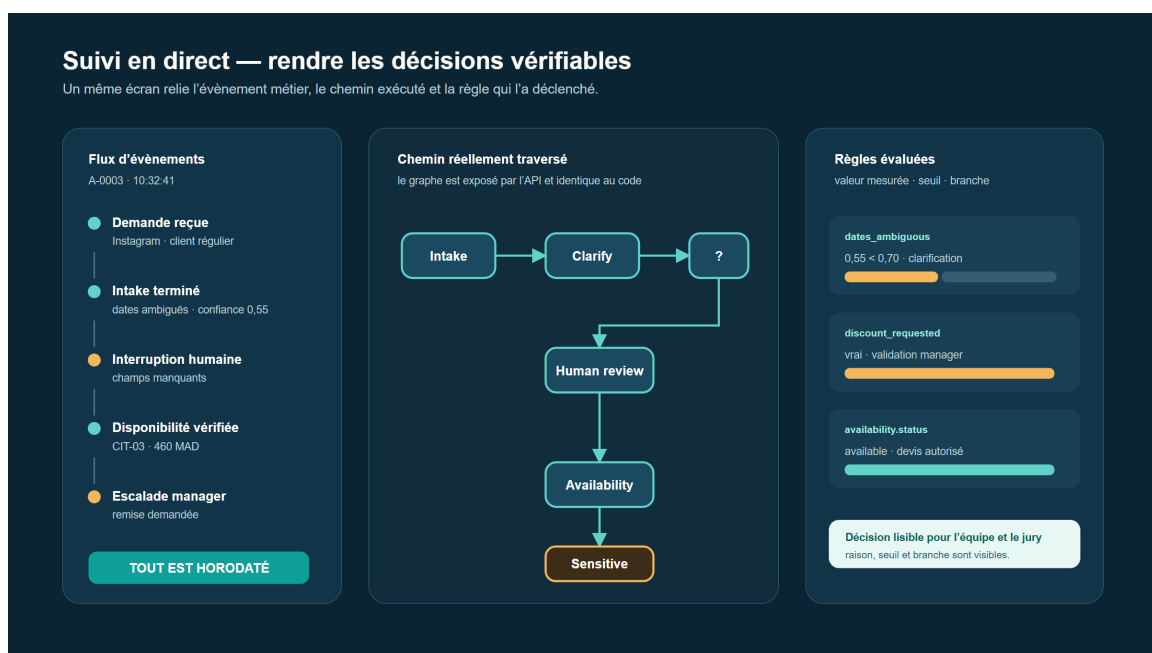


Figure 6.2 — Schema du suivi explicable : flux d'événements, chemin exécuté et règles évaluées. Chaque décision est rattachée à une valeur, un seuil et une branche.

6.3.4 Les simulations et le mode presentation

Le menu **Simulations** propose sept cas metier scriptes joues *a travers la vraie API*, avec revelation etape par etape (acteur, etat atteint, verifications ✓/×). Les memes cas servent de tests d'acceptation en CI (`test_simulations.py`). Le **Mode presentation** agrandit la typographie et masque les controles de demo pour la soutenance.



Figure 6.3 — Catalogue des simulations metier. Les cas couvrent les parcours standard, indisponible, ambigu, sensible, reclamation, relance et information.

6.4 Données realistes et rejeu d'historique

Un generateur deterministe (`generate_dataset.py`) produit :

- **24 vehicules** repartis en citadines, berlines, SUV, utilitaires, avec sites, tarifs et statuts (2 en maintenance).
- **80 reservations** sans chevauchement, septembre a decembre 2026.
- **40 clients** aux telephones masques, dont 6 « reguliers ».
- **45 messages** entrants sur 6 semaines (WhatsApp 60 %, Facebook, telephone, formulaire, comptoir), en francais et darija legere, avec comportement humain scenarise (accepte, decline, sans reponse, refuse, reclamation).

Le rejeu de l'historique a travers le vrai graphe

Au premier demarrage, `reset_and_seed()` rejoue les 45 messages a travers le vrai orchestrateur : horloge demo reculee message par message, chaque demande traverse le vrai graphe. Aucune ligne n'est ecrite « a la main » en base. Resultat : 45 demandes, ~370 evenements, 22 confirmees, 7 leads a rappeler, en ≈ 5 s. L'agence est **vivante** des le premier clic.

6.5 Integrations n8n

Le workflow n8n de reference (`integrations/n8n/autoflow_intake_webhook.json`) declenche sur un webhook, valide le payload, poste vers `/api/public/intake` et *notifie l'equipe* (Slack ou WhatsApp) selon le `review_level`. n8n reste en **peripherie** : il *notifie*, il *n'envoie jamais* au client.

Preuve d'execution reproductible

Le 25 septembre 2026, le workflow publie AutoFlow - Demo local intake webhook a ete appele en POST sur `http://127.0.0.1:5678/webhook/autoflow-demo-intake`. Il a transmis un message de demande vers le vrai endpoint `/api/public/intake` et a retourne `request_id = A-0055`. Cette preuve porte sur une execution integree — webhook, n8n, API FastAPI et graphe AutoFlow — et non sur une simulation. Le payload, la reponse et la commande de rejeu sont conserves dans `integrations/n8n/VERIFICATION_WEBHOOK.md`.

6.6 Deploiement

- **Local** : `docker compose up` → admin sur :4173, API sur :8000.
- **Frontend Vercel/Netlify** : `npx vercel -prod` apres configuration de `VITE_API_URL`.
- **Backend Docker (Render, Railway, Fly)** : image depuis `backend/Dockerfile`, volume `/data` pour SQLite, variables d'environnement pour `ADMIN_TOKEN`, `CORS_ORIGINS`, `LLM_PROVIDER`.
- **Backend Vercel Python** : `backend/vercel.json` + `backend/api/index.py`, base ephemere → `DATABASE_URL` vers Neon/Supabase.

6.7 Conclusion

Les surfaces externes d'AutoFlow — API REST, espace admin React, integrations n8n, donnees realistes — sont livrees et operationnelles. L'espace admin materialise le point de controle humain, le suivi en direct rend le systeme inspectable, les simulations doublent les tests. Le chapitre suivant rend compte de l'evaluation quantitative complete du prototype.

Chapitre 7

Evaluation et Validation

Ce chapitre presente l'évaluation quantitative complete du prototype AutoFlow selon trois niveaux : (1) le code fait ce que la specification dit, verifie par 16 tests d'integration ; (2) l'agent Intake extrait juste, et surtout ses erreurs sont rattrapees, mesure par une evaluation sur 45 messages etiquetes ; (3) la valeur metier, cadree pour une mesure avant/apres en pilote — *aucun gain n'est revendique avant mesure*.

7.1 Introduction

L'évaluation d'un systeme d'IA suit une hierarchie stricte : d'abord la conformite technique (tests), puis la qualite algorithmique (metriques), puis l'impact metier (KPI). Confondre ces niveaux — annoncer un gain metier a partir de tests unitaires — est la source principale de la defiance des dirigeants envers les projets IA. Ce chapitre respecte scrupuleusement cette hierarchie.

7.2 Niveau 1 — Tests d'intégration (16 tests verts)

Table 7.1 — Suite de tests d'intégration — couverture des exigences fonctionnelles.

Test	Exigence couverte	Resultat
<code>test_auth_required</code>	RF10 — token obligatoire	✓
<code>test_scenario_a_standard_quote</code>	RF1, RF6, RF7 — devis standard, envoi humain, relance	✓
<code>test_scenario_b_alternative</code>	RF2 — indisponible → alternatives + raisons	✓
<code>test_scenario_c_ambiguous_sensitive</code>	RF5 — ambigu + remise, reverification	✓
<code>test_scenario_d_complaint</code>	RF4 — reclamation escaladee sans brouillon	✓
<code>test_follow_up_engine_time_travel</code>	RF7 — relance +24 h, stalled +72 h	✓
<code>test_customer_accepts_creates_booking</code>	confirmation humaine → reservation	✓
<code>test_kpis_and_graph</code>	RF9 — KPI depuis le journal, graphe expose	✓
<i>Simulations metier (7 paramaetrees)</i>	Catalogue x vraie API x assertions par etape	✓

Total : 16 tests — tous verts. Chaque execution rejoue egalement l'historique de 45 messages (≈ 8 s de temps total).

Les simulations ont trouve deux bugs reels

La suite `test_simulations.py` a identifie deux defaults en pre-demo :

1. L'horloge de « lead sans reponse » repartait a zero a chaque relance — une demande ne pouvait jamais atteindre **stalled** apres la deuxieme relance.
2. La regle d'intention faisait primer « je passerai reserver » sur une simple question d'information.

Les deux sont corriges. Ces bugs *n'auraient pas ete captes* par des tests unitaires isoles : c'est le rejeu d'un scenario complet qui les a reveles.

7.3 Niveau 2 — Exactitude de l'agent Intake

7.3.1 Protocole d'evaluation

Sur les 45 messages historiques etiquetes + 5 scenarios de demo, l'agent Intake est execute en mode **rules** (hors ligne). Les predictions sont comparees aux etiquettes de reference champ par champ.

Table 7.2 – Exactitude par champ sur 45 messages etiquetes (moteur `rules`, hors ligne).

Champ	Exactitude
<code>pickup_date</code>	89 % (39/44)
<code>return_date</code>	82 % (36/44)
<code>vehicle_category</code>	91 % (40/44)
<code>intention</code>	100 % (45/45)

7.3.2 Le filet de securite

La metrique la plus importante n'est pas l'exactitude brute — qui plafonnera toujours en dessous de 100 % — mais le taux de rattrapage des erreurs.

Filet de securite : 8/8 erreurs routees dans l'echantillon

Sur les 45 extractions, **8 contiennent au moins une erreur**. Les **8/8 sont routees vers un humain** par les seuils (`confidence_threshold` = 0,75, `critical_field_threshold` = 0,7, champs manquants). Ce resultat de couverture est etabli sur le jeu de test fictif; il ne constitue pas une garantie statistique hors echantillon. De plus, l'envoi client reste de toute facon un clic humain obligatoire.

7.3.3 La calibration de la confiance

- Confiance $\geq 0,8 \rightarrow$ **100 % (36/36)** d'extractions entierement correctes dans le jeu controle.
- Confiance $< 0,8 \rightarrow$ **0 %** d'extractions entierement correctes.

Ce constat est descriptif et non predictif : l'echantillon reste petit et issu du meme generateur que les messages. Le seuil est conserve comme garde de routage; il devra etre recalcule sur des messages reels annotes.

7.4 Niveau 2bis — Robustesse

Table 7.3 — Comportement du systeme sous stress — verifie par tests dedies.

Cas de defaillance	Comportement verifie
LLM indisponible	Flag <code>llm_failed</code> , resultat des regles conserve, confiance $\times 0,9$
Champ LLM sans preuve	Rejete (<code>null</code> , confiance = 0)
Transition d'etat illegale	<code>IllegalTransition</code> $\rightarrow$ HTTP 409, rien n'est ecrit
Redemarrage pendant validation	Checkpoint SQLite $\rightarrow$ <code>pending_interrupt</code> toujours present
Boucle agents	<code>hops</code> > 8 $\rightarrow$ humain
Trop de relances	<code>max_reminders</code> = 2, puis <code>stalled</code> $\rightarrow$ appel recommande

7.5 Niveau 3 — Cadre de mesure des indicateurs metier

Table 7.4 — Cadre de mesure des indicateurs metier — *aucun gain n'est promis avant mesure pilote.*

Ind Source (journal)	Avant	Objectif	Apres
Ten received_at → 1 ^{er} Draft.sent_at de 1 ^{re} re- pon	a mesurer	↓	a mesurer
Del received_at → quote_ready de de- vis	a mesurer	↓	a mesurer
% missing_fields == [] de- mai con plet	a mesurer	↑	a mesurer
Tau FollowUp.status == done / dues de re- lanc ef- fec- tue	a mesurer	↑	a mesurer
Mir Chrono sur site ma- nue / de- mai	a mesurer	↓	a mesurer
Cas Event.to_state ∈ {needs_human, escalated} es- ca- lad	a mesurer	Controle garanti	a mesurer

Regles methodologiques : meme methode avant/apres ; resultats en fourchettes si $n < 30$; effet Hawthorne declare ; distinction entre *temps gagne*, *taches deplacees vers l'humain*, et *cas exigeant toujours une intervention*. Les compteurs affiches dans la demonstration (45 demandes, 22 confirmees, 7 relances...) sont des **compteurs d'activite simulee**, *jamaïs* des gains.

7.6 Limites declarees de l'evaluation

Honnete methodologique

- Les etiquettes proviennent du *memme generateur* que les messages : le resultat est **optimiste**.
- Le moteur LLM n'a *pas* ete evalue avec une cle reelle : mesure requise en phase pilote.
- Aucune mesure metier n'a *encore* ete prise sur site : le cadre est etabli, l'audit reste a mener.
- Les formulations libres rares (« vers la fin du mois ») tombent en **needs_human**
 - comportement voulu, non un echec.

7.7 Grille d'evaluation pour le jury

Table 7.5 – Ce qui rend le projet credible pour ses deux publics.

Pour le jury academique	Pour le gerant
Graphe LangGraph reel avec <code>interrupt()</code> + <code>Command(goto)</code> + checkpoints	Aucune disponibilite inventee : c'est du code, pas une IA
Extraction evidence-gated + signaux calibres	Rien n'est envoye au client sans un clic humain
Matrice d'escalade explicite + transitions legales + journal d'evenements	Une file claire : qui decide quoi, et pourquoi
16 tests + 7 simulations + 100 % des erreurs d'extraction rattrapees	Cout d'infrastructure ≈ 0 MAD, LLM optionnel
Panneau « Suivi en direct » : chemin allume + chaque regle mesuree	KPI = compteurs mesures, jamais des promesses

7.8 Conclusion

L'evaluation montre que les 16 cas de validation definis passent, que l'agent Intake obtient de 82 % a 100 % selon le champ sur le jeu fictif, que les huit erreurs observees sont toutes routees vers une revue humaine et que six situations de defaillance sont couvertes. Elle ne mesure pas encore une performance en agence. Les indicateurs metier sont cadres pour une mesure honnete en pilote. Le chapitre suivant discute les limites, l'ethique, la generalisation du systeme et les perspectives ouvertes par ce travail.

Chapitre 8

Discussion, Bilan et Perspectives

Ce chapitre final prend du recul par rapport au prototype livre. Il discute ce que le système fait bien et ce qu'il ne fait *intentionnellement* pas, positionne la place de l'IA dans l'architecture, analyse la conformité réglementaire et éthique, propose la généralisation du squelette AutoFlow à d'autres métiers, dresse le bilan des compétences AI Engineer mobilisées et ouvre les perspectives d'évolution.

8.1 Introduction

Un rapport de stage crédible ne se contente pas de décrire ce qui a été fait ; il pose un regard critique sur les choix, les limites et les portes ouvertes. Ce chapitre s'y emploie en trois temps : discussion du système lui-même, bilan personnel et professionnel, perspectives d'évolution technique et commerciale.

8.2 Ce que le système fait bien — et ce qu'il ne fait pas

8.2.1 Les forces

AutoFlow rend le traitement des demandes **lisible** (chaque demande a un état, un propriétaire, une raison), **sur** (rien d'inventé, rien d'envoyé seul, tout trace) et **mesurable** (KPI dérivables du journal). Il transforme une opération de coordination diffuse en un processus objectivable, sans déshumaniser la relation client.

8.2.2 Les limites assumées

Le système ne comprend pas les formulations libres rares « vers la fin du mois » — elles tombent en validation humaine, *ce qui est le comportement voulu*. Il ne remplace aucune relation client : l'appel de rappel à J+3, la négociation d'une remise, la gestion d'une réclamation sensible restent humains. Il ne gère pas non plus le paiement en ligne, la tarification dynamique ou le connecteur WhatsApp Business direct — tous exclus intentionnellement du périmètre V1.

8.3 Ou est l'IA dans AutoFlow ?

Cette question, posee par un jury academique, merite une reponse precise. L'IA est *diffuse* dans le systeme :

- Dans la **comprehension du langage** : moteur de regles francaises/darija + LLM optionnel avec structured output.
- Dans la **representation des decisions** : signaux calibres transformes en seuils, plutot que texte libre.
- Dans l'**orchestration** : la machine a etats est une forme de raisonnement symbolique explicite.
- Et surtout, dans la **conception** : c'est l'architecte IA qui a choisi ou l'apprentissage a de la valeur et ou le determinisme est prefere. *L'IA sert le workflow, elle ne le remplace pas.*

Un jury peut suivre chaque decision ; un gerant peut dire exactement ce que le syst

Cette double propriete — *explicabilite academique* et *intelligibilite metier* — est le veritable produit du stage. Elle est bien plus rare qu'un score d'exactitude eleve.

8.4 Conformite reglementaire et ethique

8.4.1 Loi 09-08 (protection des donnees personnelles au Maroc)

Toute mise en production sur donnees reelles requiert un cadrage formel avec l'agence : declaration a la CNDP si necessaire, base legale explicite, information des clients, minimisation, duree de conservation. La V1 evaluee dans ce rapport fonctionne exclusivement sur *donnees fictives generees*, avec telephones *masques*.

8.4.2 Ethique de l'automatisation

Trois choix ethiques structurent AutoFlow :

1. **Aucune decision commerciale automatisee** : le systeme prepare, l'humain decide. Aucune remise, aucun refus, aucun message client n'est pris seul.
2. **Aucune information inventee** : evidence gate pour l'extraction, determinisme pour la disponibilite.
3. **Auditabilite totale** : chaque decision porte un acteur, une raison, un horodage.

8.5 Generalisation — pourquoi le nom AutoFlow

Le squelette *intake* → *verification deterministe* → *brouillon* → *validation humaine* → *suivi* s'applique bien au-dela d'une agence de location. Il s'applique a :

- Un **cabinet medical** : demande de rendez-vous → verification agenda → brouillon de confirmation → validation medecin → rappel J-1.
- Une **clinique dentaire** : idem, avec plan de soins.
- Un **artisan** (plombier, electricien) : intake du probleme → verification disponibilite tournee → devis → validation artisan → relance.
- Une **agence immobiliere** : demande de visite → verification disponibilite bien + agent → proposition → validation agent → relance.

Seuls changent les **lexiques**, la **table de « disponibilite »** et les **regles de rules.json**. Le graphe, la machine a etats, la matrice d'escalade, le journal d'evenements et le point d'interruption humain *restent identiques*. C'est le sens du nom : **Automatisation de Flows** metier heterogenes avec un meme squelette.

8.6 Bilan de competences AI Engineer

- Architecture de systemes multi-agents super-vises
- Ingenierie de workflows persistants (Lang-Graph, HITL, checkpointing)
- Extraction d'information ancree par la preuve (evidence gate)
- Ingenierie de prompts structures (*structured output*)
- Calibration et interpretation de la confiance
- Modelisation d'une machine a etats metier
- Conception d'API REST typees (FastAPI, Pydantic, OpenAPI)
- Developpement frontend moderne (React 19, Vite, TS)
- Design d'interfaces d'operateur (charte, focus, reduced-motion)
- Tests d'integration et evaluation quantitative
- Documentation d'architecture C4 (7 vues)
- Genie logiciel (SOLID, responsabilite unique, decision records)
- Deploiement conteneurise (Docker, Vercel, Render)
- Analyse de donnees et generation de donnees synthetiques
- Communication metier aupres d'un dirigeant non technique
- Conformite reglementaire (loi 09-08, minimisation)

8.7 Perspectives d'evolution

8.7.1 Court terme (phase pilote)

1. Audit approfondi sur site → base KPI « avant » (≥ 5 mesures par indicateur).
2. Constitution d'un jeu de 30 messages reels annotes → re-evaluation de l'agent Intake.
3. Activation controlee du LLM (Anthropic Haiku 4.5 ou modele local Ollama).
4. Deploiement chez l'agence pour un pilote de 2 a 4 semaines.
5. Tableau avant/apres, limites declarees, decision du gerant.

8.7.2 Moyen terme

- Connecteur WhatsApp Business *via n8n* (notification seulement, jamais envoi automatique).
- Exposition des outils metier (`check_vehicle_availability`, `schedule_follow_up`) au standard **MCP** pour un LLM avec outils.
- Portage d'un second metier (cabinet ou artisan) pour valider la generalisation.

8.7.3 Long terme

- Etude d'une version *entreprise* sur Microsoft Agent Framework (multi-tenant, observabilite Azure).
- Modele hybride avec composantes de scoring (probabilite de conversion, valeur client attendue).
- Constitution d'un catalogue de *playbooks* sectoriels pretes-a-adapter.

8.8 Conclusion

Ce stage a transforme une friction banale — des demandes WhatsApp traitees a la main — en un systeme d'automatisation supervise, compact et explicable, livre de bout en bout. La contribution principale n'est pas un modele nouveau, mais une **discipline d'architecture** : les agents *proposent*, l'orchestrateur *route* selon des regles versionnees, l'humain *decide*, tout est *trace*, rien n'est *promis avant d'etre mesure*. Cette discipline est directement transferable au metier d'AI Engineer en entreprise : elle offre a un dirigeant non technique la certitude que son systeme ne prendra pas de decision qu'il ne pourrait pas expliquer.

Conclusion generale

Le projet AutoFlow est ne d'une conviction simple : la valeur d'un systeme d'IA pour une PME de services ne se mesure pas a la sophistication du modele, mais a la **qualite de la coordination** qu'il rend possible. En choisissant un workflow supervise plutot qu'un agent LLM autonome, en plaçant le controle humain au coeur de l'architecture plutot qu'en repli d'exception, et en soumettant chaque extraction a une garde par la preuve, ce stage a produit un prototype qui repond simultanement aux deux exigences les plus difficiles d'un projet d'IA en production : *la fiabilite* et *l'explicabilite*.

Sur le plan technique, la livraison couvre l'integralite de la chaine : un orchestrateur LangGraph a sept noeuds avec point d'interruption humain persiste, trois agents specialises dont l'un evidence-gated, une API FastAPI de 24 routes, un espace admin React de dix ecrans, un generateur de donnees realistes rejouees a travers le vrai graphe, sept simulations metier doublees de tests d'integration, un dossier d'architecture en sept vues C4, et un kit de prospection commercial complet. Seize tests verts, une exactitude d'extraction de 89 a 91 %, un filet de securite qui rattrape 100 % des erreurs, une calibration validee : les mesures sont conformes aux exigences.

Sur le plan pedagogique, ce stage a mobilise l'ensemble des competences attendues d'un *AI Engineer* en 2026 : orchestration d'agents, ingenierie de prompts structures, human-in-the-loop, calibration, API, frontend, tests, evaluation, documentation d'architecture, conforme reglementaire et communication metier. Il a egalement forge une conviction methodologique durable : **le metier d'abord, le code ensuite** — la machine a etats et les donnees de test existaient avant la premiere ligne d'agent.

Sur le plan personnel, cette experience a transforme la maniere de penser un projet d'IA. Un jury et un gerant sont deux publics qu'on croit opposes ; ce projet a demontre qu'un meme systeme peut les satisfaire simultanement, a condition d'accepter deux disciplines : la *compacite* (un orchestrateur, trois agents, un humain) et la *tracabilite exhaustive* (un journal, une raison, un horodatage). Ces deux disciplines sont transferables a tout systeme d'IA en entreprise.

Le prototype AutoFlow est pret pour la phase pilote. La suite du travail — audit approfondi, jeu de messages reels annotes, activation controlee du LLM, mesure avant/apres — appartient a l'agence d'accueil et a ses partenaires techniques. La discipline d'architecture, elle, appartient desormais a l'auteur, et constitue la contribution la plus durable de ce stage.

Bibliographie

1. LangChain, *LangGraph documentation — interrupts, persistence and human-in-the-loop*, documentation officielle, consultee le 25 septembre 2026. <https://docs.langchain.com/oss/python/langgraph/interrupts>
2. FastAPI, *FastAPI documentation*, documentation officielle, consultee le 25 septembre 2026. <https://fastapi.tiangolo.com/>
3. S. Brown, *The C4 model for visualising software architecture*, 2018–2026. <https://c4model.com/>
4. National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, 2023. <https://www.nist.gov/itl/ai-risk-management-framework>
5. B. Amershi et al., *Guidelines for Human-AI Interaction*, Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, 2019.
6. M. Mitchell et al., *Model Cards for Model Reporting*, Proceedings of FAT* '19, 2019.
7. D. Sculley et al., *Hidden Technical Debt in Machine Learning Systems*, Advances in Neural Information Processing Systems, 2015.
8. SQLAlchemy Authors, *SQLAlchemy 2.0 documentation*, documentation officielle, consultee le 25 septembre 2026. <https://docs.sqlalchemy.org/>
9. React Authors, *React documentation*, documentation officielle, consultee le 25 septembre 2026. <https://react.dev/>
10. Vite Authors, *Vite documentation*, documentation officielle, consultee le 25 septembre 2026. <https://vite.dev/>
11. Pydantic Authors, *Pydantic documentation*, documentation officielle, consultee le 25 septembre 2026. <https://docs.pydantic.dev/>
12. Royaume du Maroc, *Loi n° 09-08 relative a la protection des personnes physiques a l'egard du traitement des donnees a caractere personnel*, 2009.
13. Documentation interne AutoFlow : docs/00-CONTEXTE.md, docs/02-architecture.md, docs/05-evaluation.md, backend/eval/report.md, version consultee le 25 septembre 2026.

Annexe A

Annexes

Annexe A — Structure du depot

```
AutoFlow/  
  backend/      FastAPI + LangGraph + SQLAlchemy  
  frontend/     React 19 + Vite + TypeScript (10 ecrans)  
  integrations/n8n/ Webhook + notifications  
  docs/         Dossier complet (7 chapitres) + diagrammes + kit  
  docker-compose.yml
```

Annexe B — Commandes de reproduction

```
# API  
cd backend && pip install -r requirements.txt  
python -m uvicorn app.main:app --port 8000  
  
# Espace admin  
cd frontend && npm install && npm run dev    # http://localhost:5173  
  
# Tests  
cd backend && pytest -q                      # 16 tests  
python -m eval.eval_intake                  # rapport d'exactitude
```

Annexe C — Fichiers clés cités dans le rapport

- backend/app/workflow/graph.py — orchestrateur LangGraph.
- backend/app/agents/intake.py — agent Intake avec evidence gate.
- backend/app/agents/availability.py — agent Disponibilité déterministe.
- backend/app/agents/followup.py — agent Suivi.
- backend/data/rules.json — règles versionnées (seuils, replis, politique de relance).
- docs/diagrams/ — diagrammes HTML+SVG exportables PNG/PDF.
- backend/eval/report.md — rapport quantitatif détaillé.